

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA ELEKTROTECHNIKY A INFORMATIKY

Evidenčné číslo: FEI-104376-117218

NÁVRH IMPLEMENTÁCIE VYBRANÝCH ALGORITMOV RIADENIA

Diplomový projekt

Študijný program:	Robotika a kybernetika
Študijný odbor:	Kybernetika
Školiace pracovisko:	Ústav robotiky a kybernetiky
Školiteľ:	Ing. Marián Tárník, PhD.

Bratislava 2026

Bc. Andrii Stoliar



ZADANIE DIPLOMOVEJ PRÁCE

Študent: **Bc. Andrii Stoliar**
ID študenta: 117218
Študijný program: robotika a kybernetika
Študijný odbor: kybernetika
Vedúci práce: Ing. Marián Tárník, PhD.
Vedúci pracoviska: prof. Ing. František Duchoň, PhD.
Miesto vypracovania: Ústav robotiky a kybernetiky

Názov práce: **Návrh implementácie vybraných algoritmov riadenia**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

Práca sa zameriava na samotnú implementáciu algoritmov riadenia pričom sa predpokladá uplatnenie poznatkov z oblasti pokročilých metód automatického riadenia v rámci príslušného študijného programu. Hlavným cieľom práce je zostavenie riadiaceho systému pre vybraný laboratórny dynamický systém vrátane vlastného softvérového návrhu pre dostupný laboratórny hardvér (mikrokontroléry, PLC a procesory pre distribuovaný riadiaci systém). Pre splnenie cieľa práce je tiež potrebné adekvátne uplatňovať postupy z oblasti identifikácie systémov, numerickej simulácie a vyhodnocovania kvality riadenia systémov.

Úlohy:

1. Oboznámte sa s dostupným laboratórnym hardvérom a vypracujte stručnú technickú dokumentáciu vybraných laboratórnych dynamických systémov.
2. Vypracujte krátky prehľad riadiacich algoritmov vybraných pre následnú implementáciu.
3. Navrhnite a realizujte predmetné riadiace systémy a analyzujte kvalitu riadenia.
4. Vyhodnoťte dosiahnuté výsledky.

Termín odovzdania diplomovej práce: 15. 05. 2026
Dátum schválenia zadania diplomovej práce: 06. 11. 2025
Zadanie diplomovej práce schválil: prof. Ing. Jarmila Pavlovičová, PhD. – garantka študijného programu

Podakovanie

Na tomto mieste by som rád podakoval školiteľovi Ing. Mariánovi Tárníkovi, PhD., za odborné vedenie, cenné rady a trpezlivosť počas vypracovania tejto diplomovej práce.

Zároveň ďakujem celému Ústavu robotiky a kybernetiky za odovzdanie cenných vedomostí a skúseností, ktoré mi pomohli rozvíjať sa v oblasti riadenia a automatizácie.

Veľká vďaka patrí aj mojim priateľom, starým rodičom, mame, otcovi, sestre a mojej dáme srdca za ich obrovskú podporu, povzbudenie a pochopenie počas celého štúdia.

Andrii Stoliar
Bratislava, 2026

Anotácia diplomovej práce

Slovenská technická univerzita v Bratislave

Fakulta elektrotechniky a informatiky

Študijný odbor:	kybernetika
Študijný program:	Robotika a kybernetika
Autor:	Bc. Andrii Stoliar
Diplomová práca:	Návrh implementácie vybraných algoritmov riadenia
Školiteľ:	Ing. Marián Tárník, PhD.
Mesiac, rok odovzdania:	Máj, 2026

Kľúčové slová: Lorem ipsum dolor sit amet

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Master Thesis Abstract

Slovak University of Technology in Bratislava
Fakulty of Electrical Engineering and Information Technology

Branch of Study: Cybernetics
Study Programme: Robotics and Cybernetics
Author: Bc. Andrii Stoliar
Thesis Title: Implementation Design of Selected Control Algorithms
Supervisor: Ing. Marián Tárník, PhD.
Month, Year: May, 2026

Keywords: Lorem ipsum dolor sit amet

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Obsah

Úvod	7
1 Vstupný príklad regulácie	8
1.1 PID regulácia v uzavretej spätnej väzbe	8
1.2 Prenosové funkcie ORO a URO a zákon regulácie	8
2 Identifikácia a popis tepelného systému TS05	11
2.1 Popis tepelného systému	11
2.1.1 Popis laboratórneho fyzikálneho modelu	11
2.1.2 Statické vlastnosti tepelného systému	12
2.2 Identifikácia tepelného systému	15
2.2.1 Meranie údajov a ich spracovanie	15
2.2.2 Identifikácia systému pomocou metódy najmenších štvorcov . .	17
2.2.3 Identifikácia systému pomocou optimalizácie prenosovej funkcie	18
2.2.4 Výsledky identifikácie tepelného systému	19
2.2.5 Aproximácia identifikovaných modelov	20
3 PI regulátor pre systém TS05	22
3.1 Štruktúra PI regulátora	22
3.2 Nastavenie parametrov PI regulátora pre systém TS05	23
3.3 Implementácia PI regulátora v MATLABe	25
4 PI regulácia s prepínaním pracovných bodov	29
4.1 Prepínač pracovných bodov	29
4.1.1 Opis a schéma riadenia z použitím prepínača	29
4.1.2 Experiment s prepínaním pracovných bodov	34
5 MRAC regulácia – gradientná metóda	37
5.1 Popis algoritmu	37
5.1.1 Prevod systému do stavového priestoru	39
5.1.2 Pseudokód gradientnej MRAC regulácie (diskrétna realizácia) .	42
5.2 Implementácia MRAC v prostredí MATLAB	44
Zoznam literatúry	46

Úvod

Riadenie procesov je kľúčovou súčasťou moderných priemyselných systémov, ktoré zabezpečujú efektívnu, bezpečnú a spoľahlivú prevádzku širokého spektra technologických zariadení. S rastúcimi nárokmi na kvalitu výroby, energetickú efektívnosť a flexibilitu riadenia sa čoraz väčší dôraz kladie nielen na samotný návrh riadiacich algoritmov, ale aj na ich praktickú implementáciu v reálnych podmienkach.

Hlavným zameraním tejto práce je práve problematika implementácie riadiacich algoritmov na rôznych hardvérových a softvérových platformách. Dôraz nie je kladený výlučne na teoretický návrh regulátorov, ale najmä na spôsob ich realizácie, integrácie do riadiaceho systému a na porovnanie správania algoritmov v simulačnom prostredí a pri riadení reálneho procesu. Takýto prístup umožňuje identifikovať praktické obmedzenia, ktoré sa v čisto teoretickom návrhu často neprejavajú.

Štruktúra práce v jej aktuálnej podobe zodpovedá prirodzenému postupu riešenia úloh, ktorý bol zvolený počas samotnej realizácie experimentov. Jednotlivé časti sú preto usporiadané ako postupný proces od identifikácie systému, cez návrh a ladenie riadiacich algoritmov, až po ich overenie v simulácii a pri riadení reálneho zariadenia. Takéto členenie verne odráža praktický spôsob práce a umožňuje sledovať vývoj návrhu riadenia krok za krokom.

V konečnej verzii práce však bude štruktúra upravená tak, aby zodpovedala štandardnému členeniu technicky orientovaných prác. Najskôr bude systematicky predstavený samotný algoritmus riadenia, jeho logika a vzájomné väzby medzi jednotlivými blokmi, v prípade potreby doplnené o pseudokód. Následne bude uvedený podrobný opis riadeného systému, jeho fyzikálne vlastnosti a proces identifikácie. Až v ďalších kapitolách bude spracovaná samotná implementácia riadenia v simulačnom prostredí a na zvolenej hardvérovej platforme.

Z tohto dôvodu bude existujúca, už vypracovaná časť práce dodatočne štrukturálne upravená a reorganizovaná, pričom obsah jednotlivých kapitol zostane zachovaný. Cieľom tejto úpravy je zosúladiť výslednej štruktúry práce s požiadavkami zadania a zároveň zachovanie logickej nadväznosti medzi jednotlivými krokmi návrhu a implementácie riadenia.

1 Vstupný príklad regulácie

Začatie tejto práce bolo zvolené formou opisného úvodného príkladu regulácie, konkrétne štandardného a všeobecne známeho PID regulátora. Predpokladajme, že máme k dispozícii identifikovaný model systému, prípadne uvažujeme priamu implementáciu algoritmu na reálnom systéme. V oboch prípadoch budeme riadený objekt ďalej označovať ako $G_s(s)$.

1.1 PID regulácia v uzavretej spätnej väzbe

Prenosová funkcia PID regulátora v paralelnej forme je definovaná ako

$$C(s) = K_p + \frac{K_i}{s} + K_d s, \quad (1)$$

kde K_p predstavuje proporcionálny zisk, K_i integračný zisk a K_d derivačný zisk regulátora.

Na reguláciu budeme uvažovať štandardnú zápornú spätnú väzbu. Regulačná odchýlka vzniká ako rozdiel medzi požadovanou hodnotou a výstupom systému,

$$e(t) = w(t) - y(t), \quad (2)$$

pričom $w(t)$ je žiadaná hodnota (setpoint) a $y(t)$ je výstup regulovanej sústavy. Výstup regulátora predstavuje akčný zásah $u(t)$, ktorý vstupuje do systému $G_s(s)$.

Pre diskretnú implementáciu s periódou vzorkovania T_s budeme používať zápis

$$e(k) = w(k) - y(k), \quad (3)$$

kde index k označuje diskretný časový krok.

Bloková schéma uzavretého regulačného obvodu má nasledujúci tvar:



Obr. 1: Bloková schéma uzavretého regulačného obvodu so zápornou spätnou väzbou a PID regulátorom

1.2 Prenosové funkcie ORO a URO a zákon regulácie

Predpokladajme, že regulovaný systém má prenosovú funkciu s čitateľom nultého rádu a menovateľom druhého rádu,

$$G_s(s) = \frac{b_0}{s^2 + a_1 s + a_0}. \quad (4)$$

Prenosová funkcia otvoreného regulačného obvodu (ORO) je daná súčinom systému a regulátora,

$$G_{\text{ORO}}(s) = G_s(s) C(s). \quad (5)$$

Po dosadení (1) a (4) dostaneme

$$\begin{aligned} G_{\text{ORO}}(s) &= \frac{b_0}{s^2 + a_1 s + a_0} \left(K_p + \frac{K_i}{s} + K_d s \right) \\ &= \frac{b_0 (K_d s^2 + K_p s + K_i)}{s (s^2 + a_1 s + a_0)}. \end{aligned} \quad (6)$$

Prenosová funkcia uzavretého regulačného obvodu (URO) pri jednotkovej spätnej väzbe je

$$G_{\text{URO}}(s) = \frac{G_{\text{ORO}}(s)}{1 + G_{\text{ORO}}(s)}. \quad (7)$$

Po dosadení (6) platí

$$G_{\text{URO}}(s) = \frac{b_0 (K_d s^2 + K_p s + K_i)}{s (s^2 + a_1 s + a_0) + b_0 (K_d s^2 + K_p s + K_i)}. \quad (8)$$

Póly ORO sú dané koreňmi menovateľa prenosu $G_{\text{ORO}}(s)$, t.j.

$$s (s^2 + a_1 s + a_0) = 0, \quad (9)$$

odkiaľ priamo vyplýva jeden pól v $s_1 = 0$ a zvyšné dva póly sú koreňmi kvadratickej rovnice

$$s^2 + a_1 s + a_0 = 0. \quad (10)$$

Póly URO sú určené koreňmi charakteristického polynómu (menovateľa) uzavretého obvodu,

$$s (s^2 + a_1 s + a_0) + b_0 (K_d s^2 + K_p s + K_i) = 0, \quad (11)$$

pričom po roznásobení dostaneme explicitný tvar tretieho rádu

$$s^3 + (a_1 + b_0 K_d) s^2 + (a_0 + b_0 K_p) s + b_0 K_i = 0. \quad (12)$$

Stabilita je v oboch prípadoch kľúčová, avšak význam sa líši. V prípade ORO sa stabilita posudzuje najmä z hľadiska robustnosti a bezpečnosti návrhu regulátora. V prípade URO je stabilita nevyhnutná podmienka správnej funkcie regulácie. Pre spojitý systém stabilita znamená, že všetky póly $G_{\text{URO}}(s)$ majú zápornú reálnu časť. V opačnom prípade výstup $y(t)$ diverguje, regulačná odchýlka neklesá a systém môže byť v reálnej prevádzke nebezpečný.

Zákon riadenia

V časovej oblasti je PID zákon regulácie možné zapísať ako

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}. \quad (13)$$

Pre praktickú implementáciu v diskretnom čase budeme uvažovať základnú aproximáciu:

$$I(k) = I(k-1) + T_s e(k), \quad (14)$$

$$D(k) = \frac{e(k) - e(k-1)}{T_s}, \quad (15)$$

$$u(k) = K_p e(k) + K_i I(k) + K_d D(k). \quad (16)$$

Z dôvodu fyzikálnych obmedzení akčného zásahu sa v praxi uplatňuje saturácia

$$u(k) = \min(\max(u(k), u_{\min}), u_{\max}). \quad (17)$$

Pseudokód PID regulátora

Nižšie je uvedený ilustračný pseudokód výpočtu PID regulátora v diskretnom čase s vnútornými stavmi integrátora a pamäťou predchádzajúcej chyby.

```
1  % Inputs: w (setpoint), y (output), Ts
2  % States: I (integrator state), e_prev (previous error)
3  % Params: Kp, Ki, Kd, u_min, u_max
4
5  e = w - y;
6
7  % Integrator update
8  I = I + Ts * e;
9
10 % Derivative term
11 D = (e - e_prev) / Ts;
12
13 % PID law
14 u = Kp * e + Ki * I + Kd * D;
15
16 % Saturation
17 u = min(max(u, u_min), u_max);
18
19 % Memory update
20 e_prev = e;
```

Výpis kódu 1: Pseudokód diskretné implementácie PID regulátora (ilustračný príklad)

2 Identifikácia a popis tepelného systému TS05

V tejto kapitole sa budeme zaoberať laboratórnym modelom, ktorý bol poskytnutý pre potreby tejto diplomovej práce – tepelným systémom TS05. Cieľom kapitoly je podrobne opísať túto experimentálnu zostavu, vykonať jej identifikáciu a navrhnúť viaceré riešenia riadenia, a to ako klasické, tak aj pokročilejšie metódy.

Súčasťou práce je tiež implementácia vybraných riadiacich algoritmov na rôznych hardvérových platformách. Táto časť má umožniť porovnanie prístupov k implementácii riadenia v simulačnom prostredí MATLAB/Simulink a na vybranom mikrokontroléri, čím sa vytvorí priestor pre analýzu rozdielov v správaní systému a v samotnej realizácii algoritmov.

2.1 Popis tepelného systému

Táto kapitola sa zaoberá popisom laboratórneho modelu TS05 a jeho základnými statickými vlastnosťami. Model predstavuje fyzický systém určený na výučbu a experimentálne overovanie princípov riadenia v oblasti automatizácie a mechatroniky.

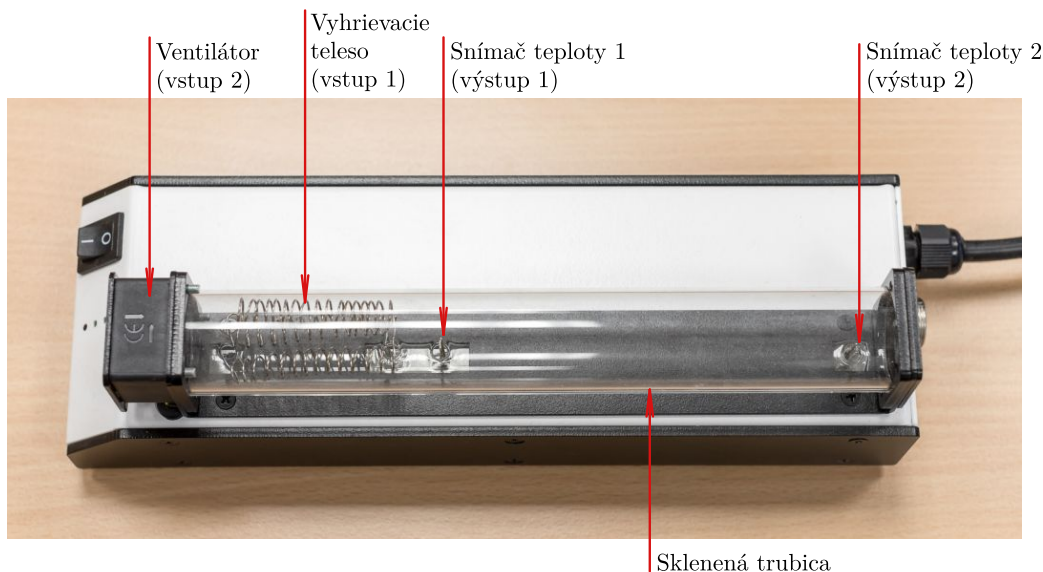
2.1.1 Popis laboratórneho fyzikálneho modelu

Laboratórny model TS05 predstavuje MIMO systém (Multiple Input – Multiple Output) so dvomi vstupmi a dvomi výstupmi, ktorého zapojenie je uvedené v tabuľke 1. Vstupné veličiny tvoria napätia riadiace ventilátor a vykurovaciú špirálu (v rozsahu 0–10 V), zatiaľ čo výstupy sú merané pomocou Senzora 1 a Senzora 2 (tiež v rozsahu 0–10 V).

Tabuľka 1: Vstupy a výstupy systému TS05

Typ signálu	Označenie	Rozsah
Vstup	Ventilátor	0–10 V
Vstup	Výkon špirály	0–10 V
Výstup	Senzor 1 (teplota 1)	0–10 V
Výstup	Senzor 2 (teplota 2)	0–10 V

Vizuálny vzhľad systému je znázornený na obrázku 2.

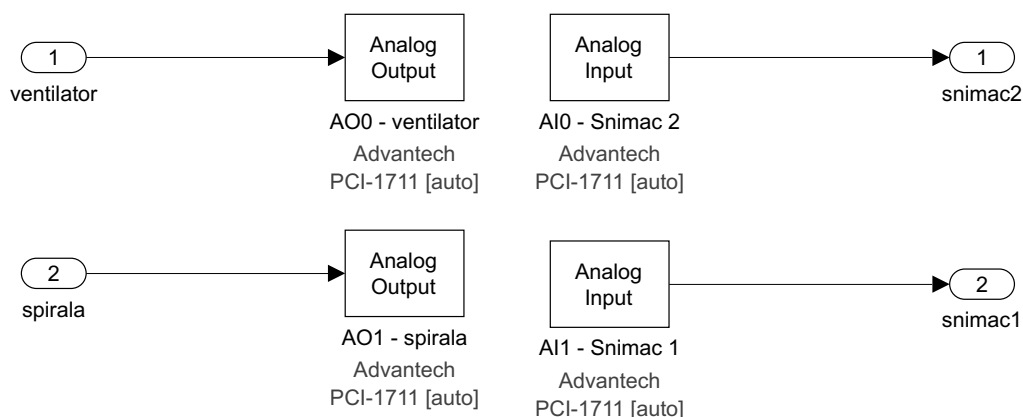


Obr. 2: Laboratórny model TS05 – reálna zostava [2]

Systém komunikuje s počítačom prostredníctvom meracej a komunikačnej karty *Humusoft*, ktorá je určená na prepojenie reálnych systémov so simulačným prostredím

MATLAB/Simulink. Táto karta umožňuje obojsmerný prenos analógových signálov – prijíma merané veličiny zo systému a zároveň vysiela riadiace signály na jeho vstupy.

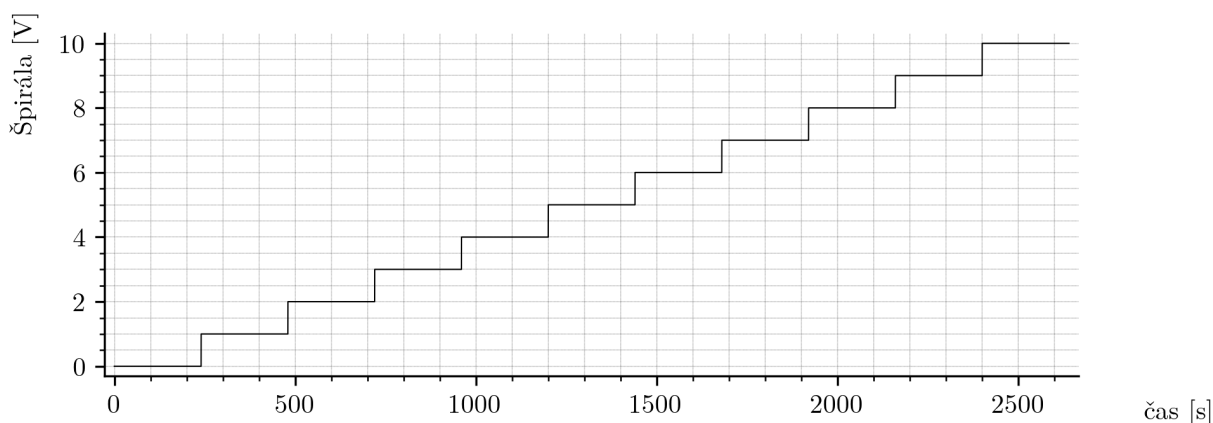
Samotná komunikácia a spracovanie signálov prebieha v prostredí Simulink, v rámci vopred pripravenej schémy (obr. 3), ktorá bola vytvorená ako súčasť projektu KUT – Krátke výučbové materiály. Táto schéma slúži aj ako podklad pre predmet *Modelovanie a riadenie systémov* [2].



Obr. 3: Simulinková schéma pre komunikáciu so systémom TS05

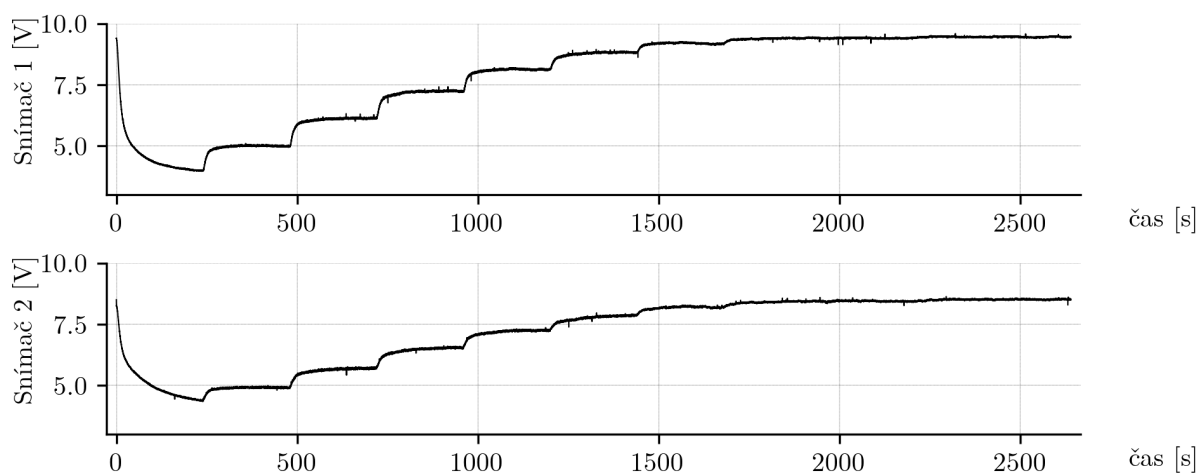
2.1.2 Statické vlastnosti tepelného systému

Na lepšie pochopenie správania systému boli experimentálne namerané jeho statické charakteristiky. Počas meraní boli skúmané tri pracovné body ventilátora – 3 V, 6 V a 9 V – pričom v každom prípade bola hodnota napätia ventilátora udržiavaná konštantná. Napätie na vykurovacej špirále sa menilo lineárne od 0 V do 10 V počas celého merania, keďže špirála má výraznejší vplyv na teplotu ako ventilátor. Priebeh signálu špirály počas experimentov je zobrazený na obrázku 4.

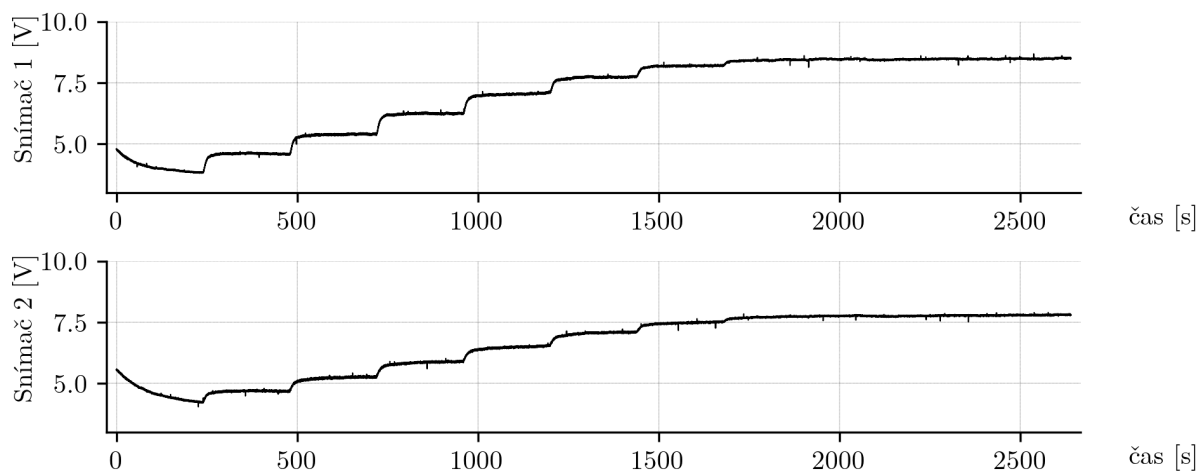


Obr. 4: Priebeh napätia na špirále počas všetkých troch experimentov

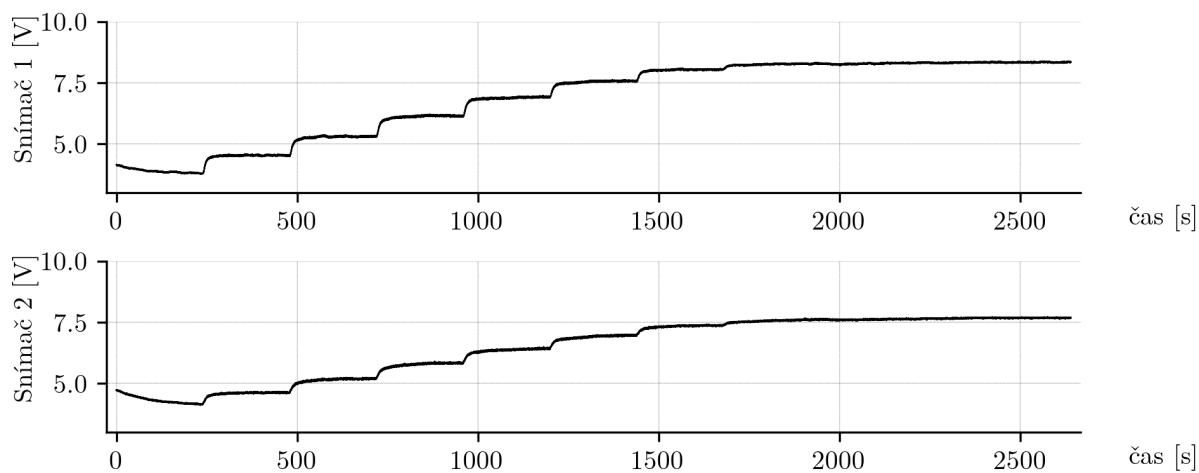
Výsledkom meraní sú šesť grafov, ktoré zobrazujú odozvu systému na zmenu vstupných veličín. Každý graf obsahuje výstupy zo Senzora 1 a Senzora 2 ako funkcie času pre jednotlivé pracovné body ventilátora.



Obr. 5: Odozva systému TS05 pre pracovný bod ventilátora 3 V

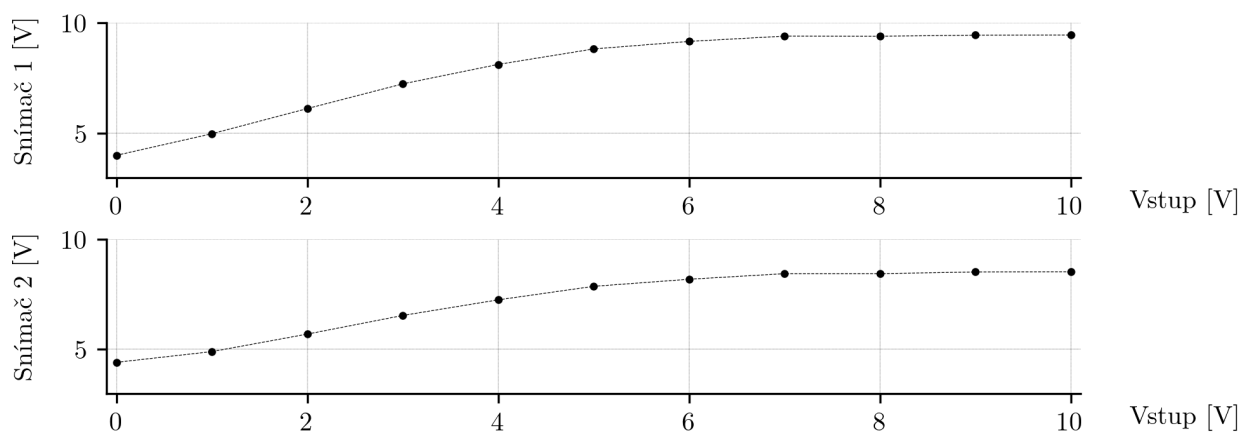


Obr. 6: Odozva systému TS05 pre pracovný bod ventilátora 6 V

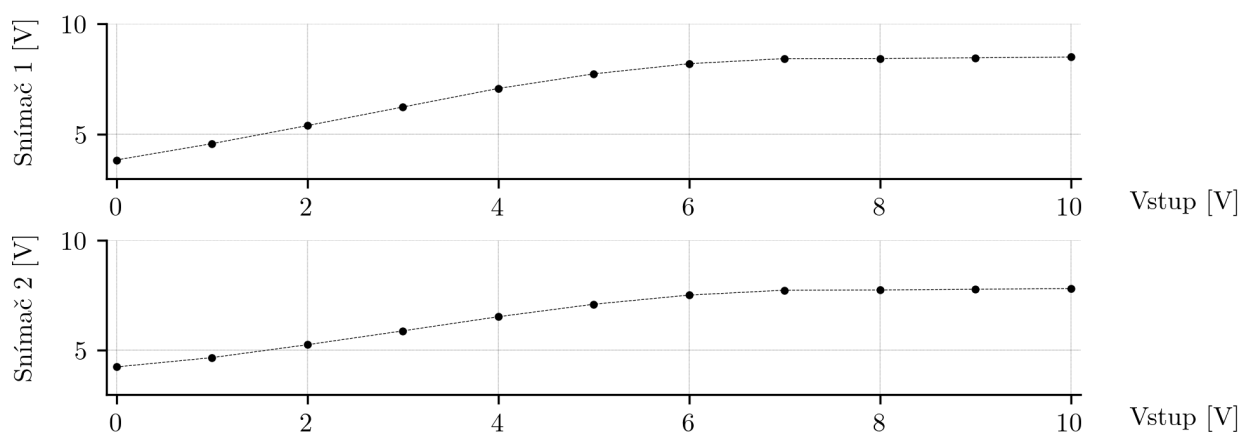


Obr. 7: Odozva systému TS05 pre pracovný bod ventilátora 9 V

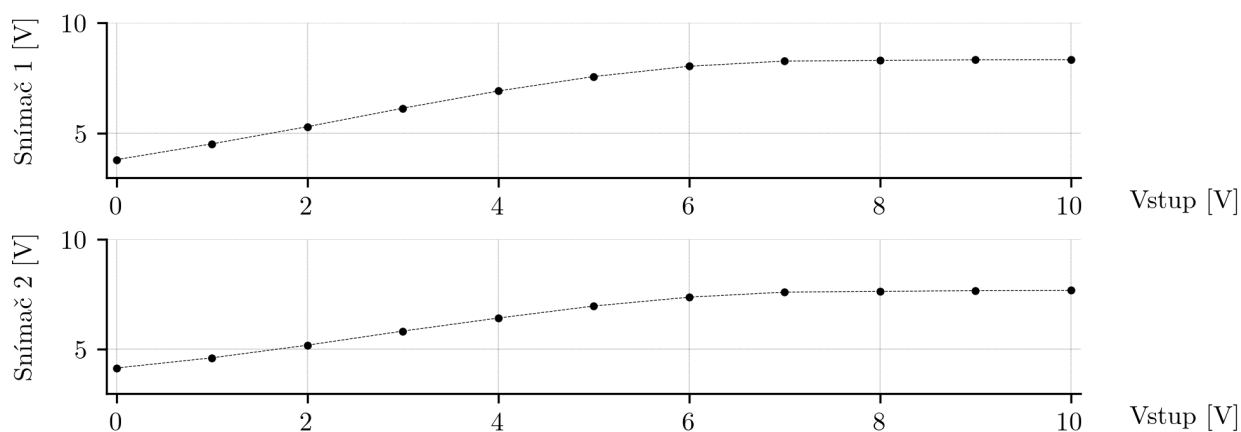
Z tých istých meraní boli zostavené aj prevodové (statické) charakteristiky, ktoré opisujú závislosť výstupných veličín od vstupného napätia špirály pre jednotlivé pracovné body ventilátora.



Obr. 8: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 3 V



Obr. 9: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 6 V



Obr. 10: Prevodová charakteristika systému TS05 pre pracovný bod ventilátora 9 V

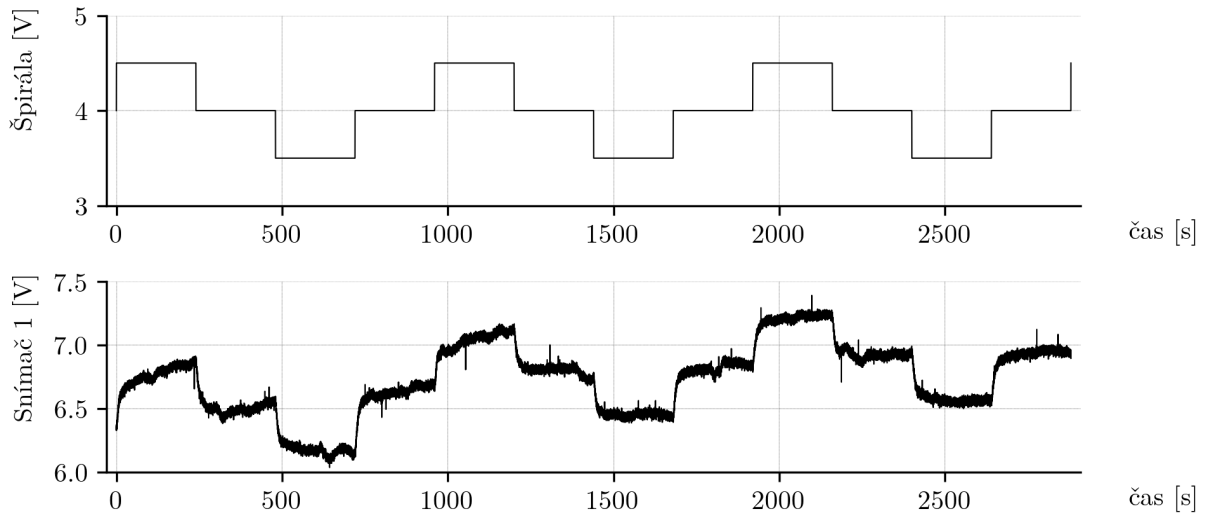
2.2 Identifikácia tepelného systému

V tejto časti sa pokúsime identifikovať tepelný systém TS05 rôznymi metódami a porovnať ich výsledky. Cieľom je určiť, ktoré prístupy poskytujú presnejšie modely a na aké faktory je potrebné dávať pozor počas procesu identifikácie.

2.2.1 Meranie údajov a ich spracovanie

Na účely identifikácie systému akoukoľvek *off-line* metódou je najprv potrebné získať reakciu systému na jednotkové skoky v okolí pracovného bodu. V našom prípade bol pracovný bod zvolený ako vstupné napätia špirály 4 V a ventilátora 6 V.

Na obrázku 11 je zobrazený vstupný signál špirály a odpoveď systému na výstupe zo senzora 1.



Obr. 11: Vstupný signál špirály a výstup senzora 1 pri skokovej zmene

Z grafu možno pozorovať zreteľný lineárny trend, teda postupné zvyšovanie výstupnej teploty počas celého merania. Tento jav je spôsobený pomalým nahrievaním špirály a tiež prirodzenými zmenami teploty v miestnosti, ktoré neboli v rámci experimentu kontrolované. Teplota prostredia sa však počas meraní pohybovala v rozsahu bežnej izbovej teploty, takže systém bol v stabilných podmienkach.

Aby bolo možné získať z nameraných údajov vhodné parametre pre identifikáciu, bolo potrebné odstrániť tento trend aplikovaním odstránenia lineárneho trendu. Metodika výpočtu je založená na lineárnej regresii, kde sa určuje priamka trendu v tvare

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x \quad (18)$$

pričom koeficienty sa počítajú podľa vzťahov [3]:

$$\hat{\beta}_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} \quad (19)$$

Takto určený trend je následne odčítaný od pôvodného signálu:

$$y_{\text{detr}} = y - (\hat{\beta}_0 + \hat{\beta}_1 x) \quad (20)$$

Uvedený postup bol implementovaný v prostredí *MATLAB* pomocou vlastnej funkcie:

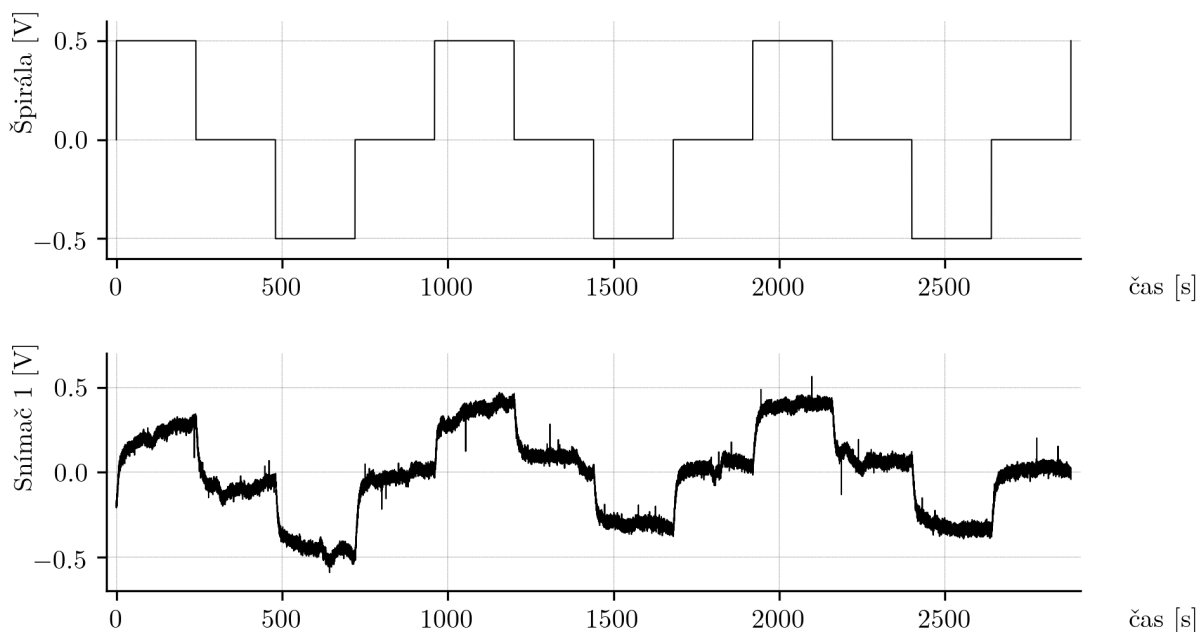
```

1 function y_detr = my_detr(y, x)
2     y = y(:);
3     x = x(:);
4     n = length(y);
5
6     Sx = sum(x);
7     Sy = sum(y);
8     Sxx = sum(x.^2);
9     Sxy = sum(x .* y);
10
11     b1 = (n*Sxy - Sx*Sy) / (n*Sxx - Sx^2);
12     b0 = mean(y) - b1 * mean(x);
13
14     y_detr = y - (b0 + b1*x);
15 end

```

Výpis kódu 2: MATLAB implementácia detrendácie signálu pomocou lineárnej regresie

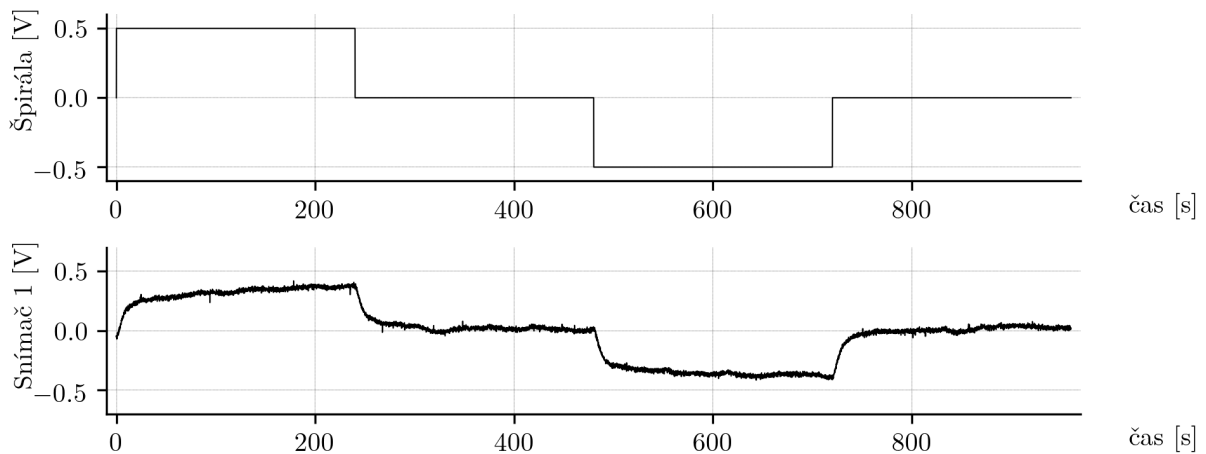
Po odstránení lineárneho trendu bol signál zároveň normalizovaný pre potreby ďalšej identifikácie. Výsledné priebehy normalizovaných a detrendovaných údajov sú zobrazené na obrázku 12.



Obr. 12: Normalizované a detrendované priebehy vstupného a výstupného signálu

Z grafov možno vidieť, že jednotlivé skokové odozvy sa mierne líšia svojím priebehom, čo poukazuje na citlivosť systému TS05 na vonkajšie podmienky a fluktuácie prostredia. Preto boli vykonané tri nezávislé experimenty s rovnakým vstupným signálom, ktorých priebehy sú zobrazené na obrázku 12. Po detrendácii a normalizácii boli tieto dáta spriemerované bod po bode, aby sa eliminoval vplyv náhodných rušení a získala reprezentatívna odozva systému.

Výsledný stredný priebeh, znázornený na obrázku 13, predstavuje typickú reakciu systému TS05 v okolí pracovného bodu 4 V s rozsahom $\pm 0,5$ V a slúži ako podklad pre ďalšiu identifikáciu modelu.



Obr. 13: Priemerná odozva systému TS05 v pracovnom bode 4 V

2.2.2 Identifikácia systému pomocou metódy najmenších štvorcov

V tejto časti bola uskutočnená identifikácia systému TS05 na základe priemernej odozvy získanej z troch experimentálnych meraní popísaných v predchádzajúcej kapitole. Tento priemerný priebeh reprezentuje typickú reakciu systému v okolí pracovného bodu 4 V a slúži ako základ pre odhad parametrov modelu. Cieľom bolo získať lineárny model druhého rádu, ktorý dostatočne presne opisuje dynamické správanie systému v tejto oblasti.

Identifikácia bola realizovaná pomocou metódy najmenších štvorcov [1], ktorá umožňuje odhad parametrov diskrétného modelu systému na základe meraných vstupov a výstupov. Pre odhad parametrov bola vytvorená regresná matica $H(k, :)$ a vektor výstupov Y , pričom výsledný odhad parametrov $\hat{\theta}$ bol určený podľa rovníc:

$$H(k, :) = [-y(k-1), -y(k-2), u(k-1), u(k-2)] \quad (21)$$

$$\hat{\theta} = (H^T H)^{-1} H^T Y \quad (22)$$

Na základe týchto odhadov boli vypočítané koeficienty diskrétného modelu druhého rádu, ktorý následne predstavuje prenosovú funkciu $G_d(z)$. Pre účely simulácie a ďalšieho návrhu riadenia bol tento model prevedený na spojitú formu $G(s)$ pomocou Tustinovej aproximácie.

V prostredí MATLAB bola implementovaná funkcia, ktorá realizuje uvedené výpočty a vytvára identifikovaný model systému. Implementácia tejto funkcie je uvedená nižšie:

```

1 function G = ls_identify_basic(y, u, Ts)
2
3     y = y(:);
4     u = u(:);
5     N = length(y);
6     na = 2; nb = 2;
7
8     H = zeros(N-2, na+nb);
9     for k = 3:N
10         H(k-2,:) = [-y(k-1), -y(k-2), u(k-1), u(k-2)];
11     end
12     Y = y(3:N);
13
14     theta = pinv(H) * Y;
15
16     a1 = theta(1);
17     a2 = theta(2);

```

```

18     b1 = theta(3);
19     b2 = theta(4);
20
21     num = [b1 b2];
22     den = [1 a1 a2];
23     Gd = tf(num, den, Ts);
24
25     G = d2c(Gd, 'tustin');
26
27 end

```

Výpis kódu 3: MATLAB implementácia identifikácie metódou najmenších štvorcov

2.2.3 Identifikácia systému pomocou optimalizácie prenosovej funkcie

V tejto časti bol zvolený menej akademický, skôr prakticky orientovaný prístup k identifikácii systému TS05. Namiesto priameho odhadu parametrov modelu metódou najmenších štvorcov bola použitá optimalizácia parametrov prenosovej funkcie tak, aby sa minimalizovala chyba medzi simulovanou odozvou modelu a reálnou priemernou odozvou systému, získanou z experimentov.

Ako nástroj optimalizácie bola použitá funkcia `fminsearch` z prostredia MATLAB, ktorá implementuje Nelderovu–Meadovu simplexovú metódu. Ide o bezderivačnú lokálnu optimalizačnú metódu, ktorá na základe hodnotenia účelovej funkcie v niekoľkých bodoch parametrov iteratívne posúva simplex (mnohouholník v priestore parametrov) tak, aby sa znižovala hodnota účelovej funkcie. V našom prípade je účelová funkcia definovaná ako koreň zo strednej kvadratickej chyby (RMSE) medzi nameranou odozvou systému a simulovaným výstupom modelu:

$$J(\mathbf{p}) = \sqrt{\frac{1}{N} \sum_{k=1}^N (y(k) - y_{\text{sim}}(k, \mathbf{p}))^2} \quad (23)$$

kde $y(k)$ je nameraný výstup, $y_{\text{sim}}(k, \mathbf{p})$ je výstup simulovaný modelom s parametrami $\mathbf{p}$ a N je počet vzoriek merania.

Optimalizácia prebieha nad parametrami diskrétného modelu druhého rádu, ktoré sú následne prevedené na spojitú prenosovú funkciu pomocou Tustinovej aproximácie. Implementácia použitej funkcie v prostredí MATLAB je uvedená nižšie:

```

1 function G = tf_identify_fmin(y, u, time, Ts)
2
3     y = y(:);
4     u = u(:);
5     time = time(:);
6
7     p0 = [0 0 0 0];
8
9     cost_fun = @(p) rmse_cost_stable(p, u, y, time, Ts);
10
11     opts = optimset('Display', 'off', 'TolX', 1e-8, 'TolFun', 1e-8);
12     p_opt = fminsearch(cost_fun, p0, opts);
13
14     p_opt = 50 * tanh(p_opt);
15     a1 = p_opt(1); a2 = p_opt(2); b1 = p_opt(3); b2 = p_opt(4);
16
17     Gd = tf([b1 b2], [1 a1 a2], Ts);
18     G = d2c(Gd, 'tustin');
19
20 end
21
22 function err = rmse_cost_stable(p, u, y, t, Ts)
23
24     p = 50 * tanh(p);
25     a1 = p(1); a2 = p(2); b1 = p(3); b2 = p(4);
26
27     try
28
29         Gd = tf([b1 b2], [1 a1 a2], Ts);

```

```

30
31     poles = pole(Gd);
32     if any(abs(poles) >= 1)
33         err = 1e6;
34         return;
35     end
36
37     G = d2c(Gd, 'tustin');
38     y_sim = lsim(G, u, t);
39
40     err = sqrt(mean((y - y_sim).^2));
41
42     catch
43         err = 1e6;
44     end
45 end

```

Výpis kódu 4: MATLAB implementácia identifikácie pomocou optimalizácie prenosovej funkcie

Pri tomto prístupe je dôležité upozorniť na jeden podstatný problém. Ak nie sú parametre prenosovej funkcie počas optimalizácie vhodne obmedzené, algoritmus môže nájsť také hodnoty parametrov, ktoré síce veľmi dobre aproximujú konkrétnu meranú odozvu, ale vedú k nerealistickému alebo nestabilnému správaniu modelu pri iných vstupoch. V predchádzajúcich pokusoch viedlo práve toto k prenosovým funkciám s „nezdravými“ parametrami, ktoré boli použiteľné iba pre konkrétny fitovaný experiment.

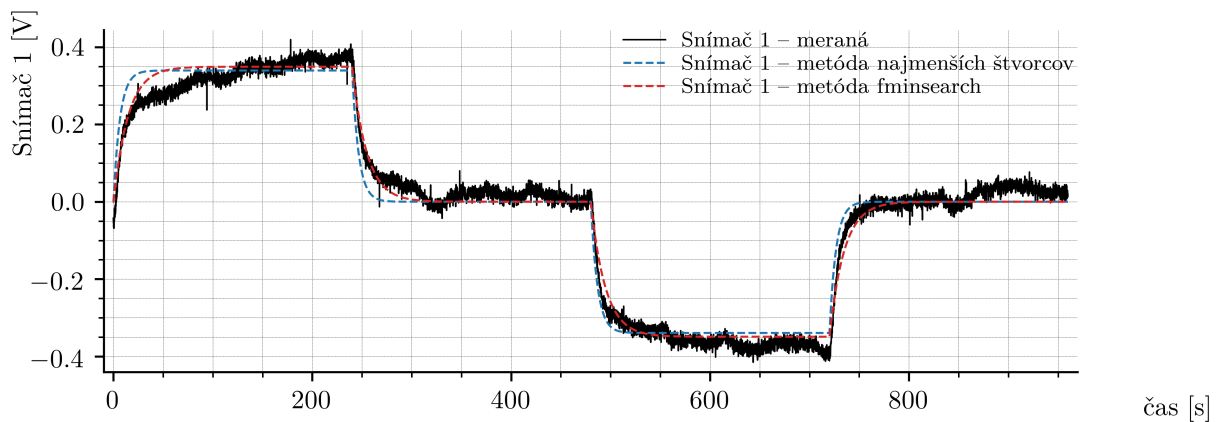
Z tohto dôvodu sú v uvedenej implementácii parametre počas optimalizácie mapované pomocou transformácie

$$p_i^* = 50 \tanh(p_i) \quad (24)$$

čím sú efektívne udržiavané v konečnom rozsahu a znižuje sa riziko vzniku numericky extrémnych alebo fyzikálne nezmyselných modelov. Okrem toho bola do optimalizácie pridaná aj dodatočná penalizácia, ktorá výrazne zvyšuje hodnotu účelovej funkcie v prípade, že identifikovaná prenosová funkcia vykazuje známky nestability. Týmto spôsobom algoritmus uprednostňuje iba stabilné riešenia a zabraňuje vzniku modelov so „zdravotne“ nevhodnými pólmi.

2.2.4 Výsledky identifikácie tepelného systému

Po identifikácii systému v pracovnom bode pomocou dvoch rôznych metód (metódy najmenších štvorcov a optimalizácie prenosovej funkcie) boli získané nasledujúce výsledky.



Obr. 14: Porovnanie skokovej odozvy reálneho systému TS05 a identifikovaných modelov

Porovnanie priemernej skokovej odozvy reálneho systému a odoziev oboch identifikovaných modelov je zobrazené na obrázku 14.

Identifikovaný model získaný metódou najmenších štvorcov má tvar

$$G_{s,LS}(s) = \frac{0.00171 s^2 - 0.3379 s + 6.074}{s^2 + 59.01 s + 8.95} \quad (25)$$

zatým čo model identifikovaný pomocou optimalizácie parametrov prenosovej funkcie metódou *fminsearch* je

$$G_{s,fmin}(s) = \frac{0.0007181 s^2 - 0.05942 s + 0.9012}{s^2 + 18.94 s + 1.291} \quad (26)$$

Kvalita zhody medzi modelmi a experimentálnymi dátami bola hodnotená pomocou ukazovateľa RMSE:

$$RMSE_{LS} = 0.036792, \quad (27)$$

$$RMSE_{fmin} = 0.026996 \quad (28)$$

Ako je vidieť, oba modely poskytujú veľmi uspokojivú aproximáciu dynamiky systému v okolí zvoleného pracovného bodu, pričom model získaný metódou *fminsearch* dosahuje o niečo menšiu hodnotu RMSE. Treba však zdôrazniť, že oba identifikované modely sú len druhého rádu, čo predstavuje rozumný kompromis medzi jednoduchosťou a presnosťou pre následný návrh regulácie.

2.2.5 Aproximácia identifikovaných modelov

Na základe získaných prenosových funkcií možno pozorovať, že ich čitatele obsahujú veľmi malé koeficienty pri prvom a druhom rade. Prítomnosť týchto členov je s veľkou pravdepodobnosťou len výsledkom numerických „artefaktov“ vzniknutých pri prevode z diskkrétnej prenosovej funkcie na spojitú formu. Zahrnutie týchto vyšších rádov v čitateli by výrazne komplikovalo niektoré analytické metódy návrhu regulátorov, napríklad metódu umiestnenia pólov.

Pre účely ďalšej práce sme sa preto rozhodli tieto prenosové funkcie aproximovať jednoduchšou formou. Najzrejmější prístup spočíva v tom, že malé koeficienty pri s a s^2 v čitateli zanedbáme a overíme vplyv tejto úpravy pomocou simulácie.

Výsledkom tejto aproximácie sú nasledujúce prenosové funkcie:

$$G_{s,LS,approx}(s) = \frac{6.074}{s^2 + 59.01 s + 8.95} \quad (29)$$

póly:

$$s_1 = -58.8550, \quad s_2 = -0.1521 \quad (30)$$

a

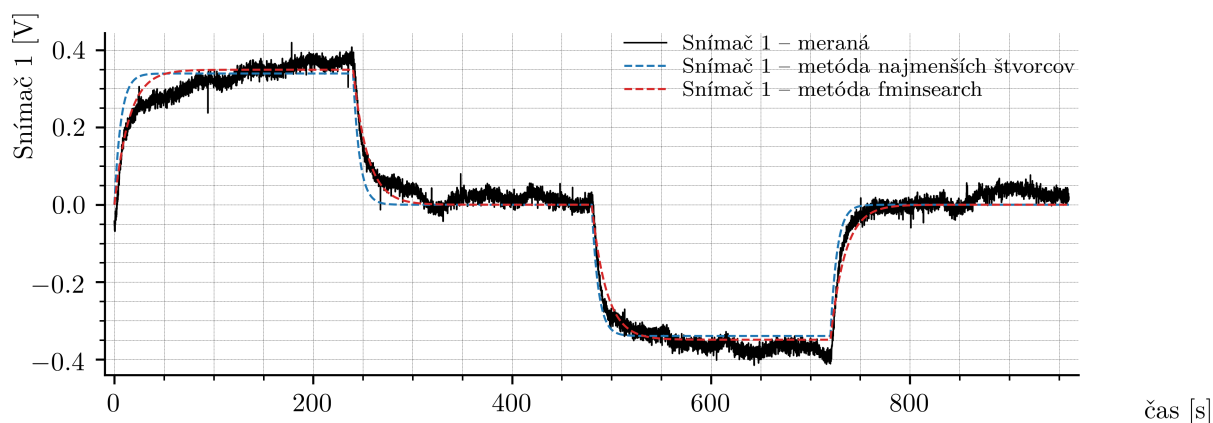
$$G_{s,fmin,approx}(s) = \frac{0.9012}{s^2 + 18.94 s + 1.291} \quad (31)$$

póly:

$$s_1 = -18.8736, \quad s_2 = -0.0684 \quad (32)$$

Z uvedených hodnôt pólov je zrejmé, že obidva modely majú póly s negatívnymi reálnymi časťami, teda ide o stabilné systémy.

Následne bola uskutočnená simulácia odozvy systému analogická tej, ktorá bola uvedená na obrázku 14. Porovnanie reálnej odozvy systému TS05 s oboma aproximovanými modelmi je zobrazené na obrázku 15.



Obr. 15: Porovnanie skokovej odozvy reálneho systému TS05 a aproximovaných modelov

Výsledné hodnoty strednej kvadratickej chyby (RMSE) medzi reálnym systémom a simulovanými odozvami boli:

$$\text{RMSE}_{\text{LS,approx}} = 0.036951, \quad (33)$$

$$\text{RMSE}_{\text{fmin,approx}} = 0.026998 \quad (34)$$

Z výsledkov, ako aj z priebehov simulácií, je zrejmé, že rozdiel medzi pôvodnými a aproximovanými modelmi je zanedbateľný. Hodnota RMSE sa zhoršila len minimálne, čo je z pohľadu ďalšieho použitia modelov úplne prijateľné, najmä vzhľadom na zjednodušenie analytického spracovania, ktoré táto aproximácia prináša.

3 PI regulátor pre systém TS05

V tejto kapitole je riešený návrh a implementácia PI regulátora ako základného nástroja pre reguláciu tepelnej sústavy. Najskôr je stručne zhrnutá teoretická štruktúra PI regulátora a jeho zapojenie v uzavretom regulačnom obvode, následne je tento prístup aplikovaný na systém TS05 vrátane ladenia parametrov a realizácie v reálnom čase.

3.1 Štruktúra PI regulátora

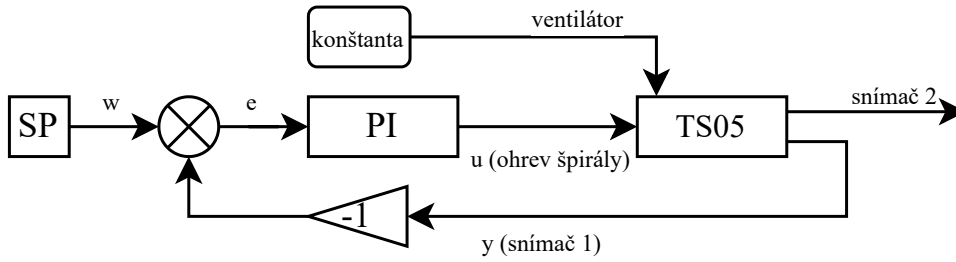
PI regulátor predstavuje zjednodušenú formu PID regulátora, v ktorej je derivačná zložka vynechaná. Pre systémy s pomalou dynamikou, ako sú tepelné sústavy, derivačný člen spravidla neprináša zlepšenie regulácie a naopak môže zosilňovať merací šum.

Paralelná forma PI regulátora je definovaná prenosovou funkciou

$$C(s) = K_p + \frac{K_i}{s}, \quad (35)$$

kde K_p je proporcionálny zisk a K_i integračný zisk regulátora.

Regulátor je zapojený v štandardnej uzavretej spätnej väzbe so zápornou spätnou väzbou. Bloková schéma uzavretého regulačného obvodu je znázornená na obr. 16.



Obr. 16: Bloková schéma uzavretého regulačného obvodu s PI regulátorom

Prenosy regulačného obvodu

Nech $G_s(s)$ označuje prenos regulovaného procesu a $C(s)$ prenos regulátora. Prenos otvoreného regulačného obvodu je potom daný vzťahom

$$G_{ORO}(s) = C(s) G_s(s), \quad (36)$$

pričom prenos uzavretého regulačného obvodu má tvar

$$G_{URO}(s) = \frac{C(s) G_s(s)}{1 + C(s) G_s(s)}. \quad (37)$$

Stabilita uzavretého regulačného obvodu je daná koreňmi charakteristickej rovnice

$$1 + C(s) G_s(s) = 0. \quad (38)$$

Pre spojitý systém je podmienkou stability, aby všetky póly uzavretého obvodu mali zápornú reálnu časť.

Zákon riadenia

V časovej oblasti možno PI regulátor zapísať v tvare

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau, \quad (39)$$

kde $e(t) = w(t) - y(t)$ predstavuje regulačnú odchýlku, $w(t)$ žiadanú hodnotu a $y(t)$ výstup systému.

Pre diskretnú realizáciu s periódou vzorkovania T_s je integračný člen aproximovaný rekurentným vzťahom

$$e(k) = w(k) - y(k), \quad (40)$$

$$I(k) = I(k-1) + T_s e(k), \quad (41)$$

$$u(k) = K_p e(k) + K_i I(k). \quad (42)$$

Z dôvodu fyzikálnych obmedzení akčného zásahu je v praxi uplatnená saturácia riadiaceho signálu.

Pseudokód PI regulátora

```

1 % Inputs: w (setpoint), y (output), Ts
2 % States: I (integrator state)
3 % Params: Kp, Ki, u_min, u_max
4
5 e = w - y;
6
7 % Integrator update
8 I = I + Ts * e;
9
10 % PID law
11 u = Kp * e + Ki * I;
12
13 % Saturation
14 u = min(max(u, u_min), u_max);

```

3.2 Nastavenie parametrov PI regulátora pre systém TS05

Pre získanie parametrov regulátora bola využitá funkcia `pidentune` v prostredí MATLAB, ktorá vykonáva automatizované ladenie regulátora. Tento postup bol zvolený preto, že viaceré analytické metódy ladenia nie je možné na systém TS05 aplikovať.

Charakteristický polynóm uzavretého regulačného obvodu pre PI regulátor má po dosadení tvar

$$s^3 + \frac{b_1}{b_2}s^2 + \left(\frac{b_0}{b_2} + \frac{a_0 K_p}{b_2}\right)s + \frac{a_0 K_i}{b_2} = 0 \quad (43)$$

Ak by sme chceli použiť metódu umiestnenia pólov pomocou rozšíreného želaného polynómu tretieho rádu, jeho všeobecný tvar je

$$s^3 + (p + 2k\omega_0)s^2 + (2k\omega_0 p + \omega_0^2)s + \omega_0^2 p = 0 \quad (44)$$

Porovnaním koeficientov polynómov (43) a (44) dostaneme sústavu rovníc:

$$1 = 1 \quad (45)$$

$$p + 2k\omega_0 = \frac{b_1}{b_2} \quad (46)$$

$$2k\omega_0 p + \omega_0^2 = \frac{b_0}{b_2} + \frac{a_0 K_p}{b_2} \quad (47)$$

$$\omega_0^2 p = \frac{a_0 K_i}{b_2} \quad (48)$$

Táto sústava nemá realizovateľné riešenie pre parametre K_p a K_i . Dôvodom je, že členy závislé od K_p a K_i sa vyskytujú iba v dvoch rovniciach, zatiaľ čo parametre želaného dynamického správania k , ω_0 , p ovplyvňujú tri rovnice. Sústava je preto pre fyzikálne konzistentné riešenie preurčená a neexistuje v nej žiadna kombinácia parametrov, ktorá by spĺňala všetky rovnice súčasne.

Rovnako nie je možné použiť metódu Ziegler–Nichols, keďže námi identifikovaný model (29) nemá kritický bod, ktorý je pre túto metódu potrebný.

Keďže návrh PI regulátora nie je hlavnou témou práce, bol zvolený numerický nástroj `pdtune`. Jeho princíp spočíva v tom, že pre zvolenú požadovanú šírku pásma uzavretého systému vypočíta optimálne hodnoty parametrov PI regulátora tak, aby bola zabezpečená dobrá dynamická odozva a primeraná robustnosť.

Najskôr bol systém prevedený do diskkrétnej podoby:

```
1 Gs = Gs_fmin;
2 Ts = 0.1;
3
4 Gz = c2d(Gs, Ts, 'tustin');
5 [Ad,Bd,Cd,Dd] = ssdata(Gz);
```

Následne bolo vykonané ladenie PI regulátora:

```
1 wc = 0.5;
2 Cz = pdtune(Gz, 'pi', wc);
3
4 [numC,denC] = tfdata(Cz,'v');
5 [Ac,Bc,Cc,Dc] = tf2ss(numC,denC);
```

Získané parametre regulátora:

$$K_p = 3.58, \quad K_i = 1.84$$

Prenosová funkcia otvoreného regulačného obvodu je

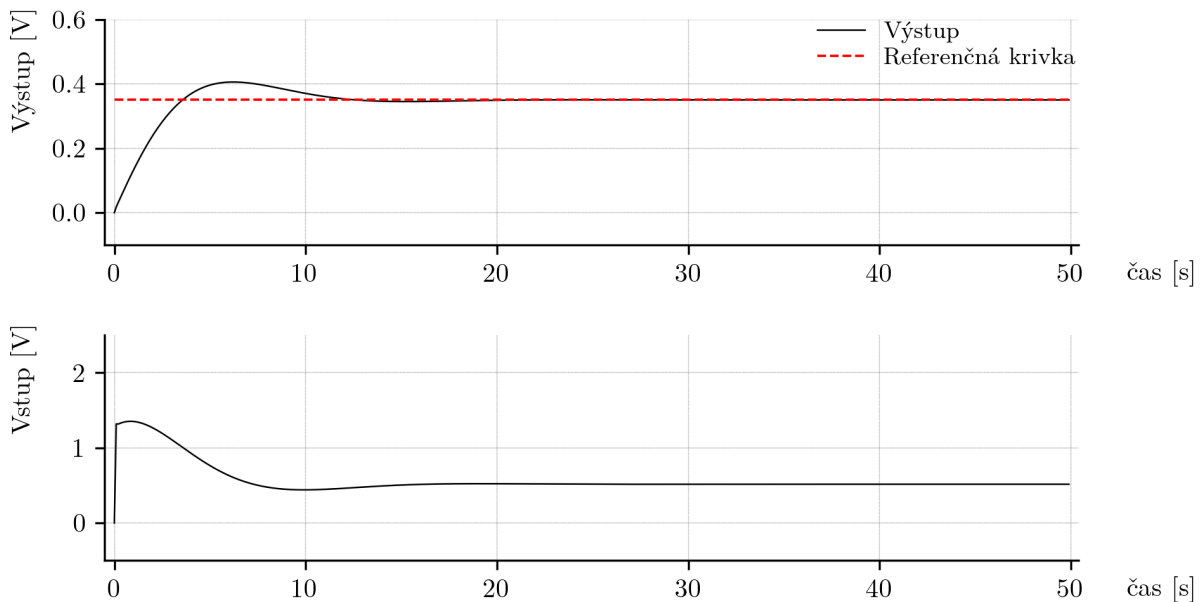
$$G_{oro}(s) = \frac{21.21s + 11.15}{s^3 + 59.01s^2 + 8.95s} \quad (49)$$

Póly otvoreného systému sú:

$$s_1 = 0, \quad s_2 = -58.855, \quad s_3 = -0.1521 \quad (50)$$

Znakom stability je prítomnosť pólov s negatívnou reálnou časťou, čo je splnené.

Simulácia bola vykonaná s obmedzením akčného zásahu na interval $[-4, 6]$, čo zodpovedá pracovnému bodu 4 V. Výsledok simulácie je uvedený na obrázku 17.



Obr. 17: Simulačný výsledok uzavretého PI riadenia systému TS05

Z výsledkov boli namerané tieto hodnoty:

$$\text{Čas ustálenia} = 11.5 \text{ s} \quad (51)$$

$$\text{Prekmit} = 15.9\% \quad (52)$$

$$\text{MSE} = 0.04369 \quad (53)$$

Dosiahnutý výsledok možno považovať za vhodný pre riadenie tepelného systému. Cieľom je dosiahnuť čo najrýchlejšiu odozvu. Prekmit približne 16 percent je akceptovateľný, keďže riadený skok je malý a tepelná sústava je pomalá, čo prirodzene limituje tvar dynamickej odozvy.

3.3 Implementácia PI regulátora v MATLABe

Ako je uvedené v zadaní, cieľom práce je implementácia riadiacich algoritmov na rôznych hardvérových a softvérových platformách. Preto bol PI regulátor implementovaný tak, aby mohol pracovať v reálnom čase bez použitia hotových Simulink blokov určených na riadenie. Celá logika riadenia bola zapísaná priamo do bloku MATLAB Function, ktorý sa vykonáva raz za každý riadiaci cyklus s periódou vzorkovania 0.1 sekundy.

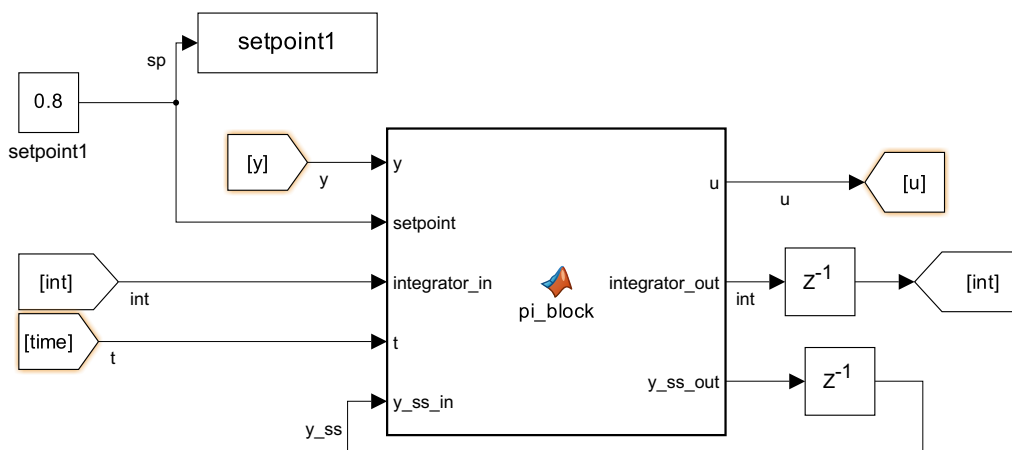
Implementácia regulátora je nasledovná:

```
1 function [u, integrator_out, y_ss_out] = ...  
2     pi_block(y, setpoint, integrator_in, t, y_ss_in)  
3  
4 % PI controller  
5  
6 pb_u = 4;  
7 Ts = 0.1;  
8 u_min = - pb_u;  
9 u_max = 10 - pb_u;  
10  
11 Kp = 3.58;  
12 Ki = 1.84;  
13  
14 % waiting phase for steady-state detection  
15 if t < 10*60  
16     u = 4;  
17     y_ss_out = y;  
18     integrator_out = 0;  
19  
20 else  
21     y_ss_out = y_ss_in;  
22     pb_y = y_ss_in;  
23  
24     % output normalization  
25     y = y - pb_y;  
26  
27     % control error  
28     e = setpoint - y;  
29  
30     % integrator  
31     integrator_out = integrator_in + e * Ts;  
32  
33     % PI control law  
34     u = Kp * e + Ki * integrator_out;  
35     u = min(max(u, u_min), u_max);  
36  
37     % input normalization  
38     u = u + pb_u;  
39  
40 end  
41 end
```

Tento blok však na rozdiel od štandardných Simulinkových riadiacich blokov neobsahuje vlastnú vnútornú pamäť, preto je potrebné stavové premenné prenášať medzi jednotlivými volaniami bloku explicitne.

Z tohto dôvodu sú hodnoty integrátora a odhad ustáleného stavu odosielané na výstup PI bloku a v nasledujúcom kroku sú privádzané späť na vstup. Týmto spôsobom sa zabezpečí, že regulátor si zachováva svoje vnútorné stavy z jedného riadiaceho kroku do druhého.

Logika zapojenia bloku MATLAB Function je zobrazená na obrázku 18, ktorý ukazuje vstupy, výstupy a spätnú väzbu stavových premenných potrebných pre správnu činnosť regulátora.



Obr. 18: Zapojenie PI regulátora v bloku MATLAB Function so spätnou väzbou stavov

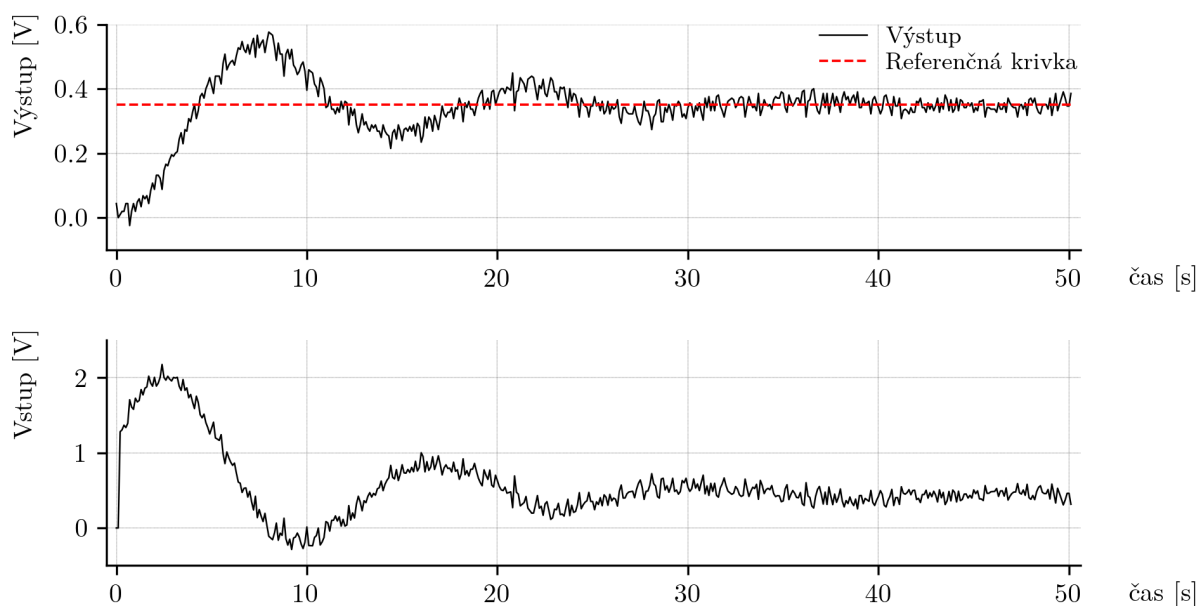
Pred samotným začiatkom regulácie bol zavedený časový úsek, počas ktorého regulátor nevykonáva žiadny zásah. Počas prvých desiatich minút sa na výstupe drží hodnota pracovného bodu a zároveň sa sleduje teplota na výstupe systému. Cieľom je získať reálnu hodnotu ustáleného stavu priamo z aktuálnych podmienok experimentu.

Použitie hodnôt zosadených zo statických charakteristík (obr. 9) nie je vhodné. Tepelný systém je citlivý na teplotu v miestnosti a experimenty nebolo možné realizovať v rovnakých podmienkach. Okrem toho surové merania (obr. 11) ukázali, že teplota narastá veľmi pomaly aj počas dlhého experimentu, čo znamená, že systém nemá pevnú, opakovateľnú hodnotu ustáleného stavu.

Z toho dôvodu je ustálený stav zisťovaný automaticky priamo regulátorom ako posledná nameraná hodnota y pred spustením PI riadenia. Tento prístup funguje bez ohľadu na to, či je systém iba málo zahriaty alebo už dosiahol vysokú prevádzkovú teplotu.

Experiment 1: Regulácia okolo jedného pracovného bodu

Na základe implementácie uvedenej vyššie bol následne vykonaný experiment v simulačnom prostredí s cieľom overiť kvalitu regulácie. Priebeh regulácie je zobrazený na obrázku 19.



Obr. 19: Simulačný experiment s PI regulátorom pri skokovej zmene žiadanej hodnoty

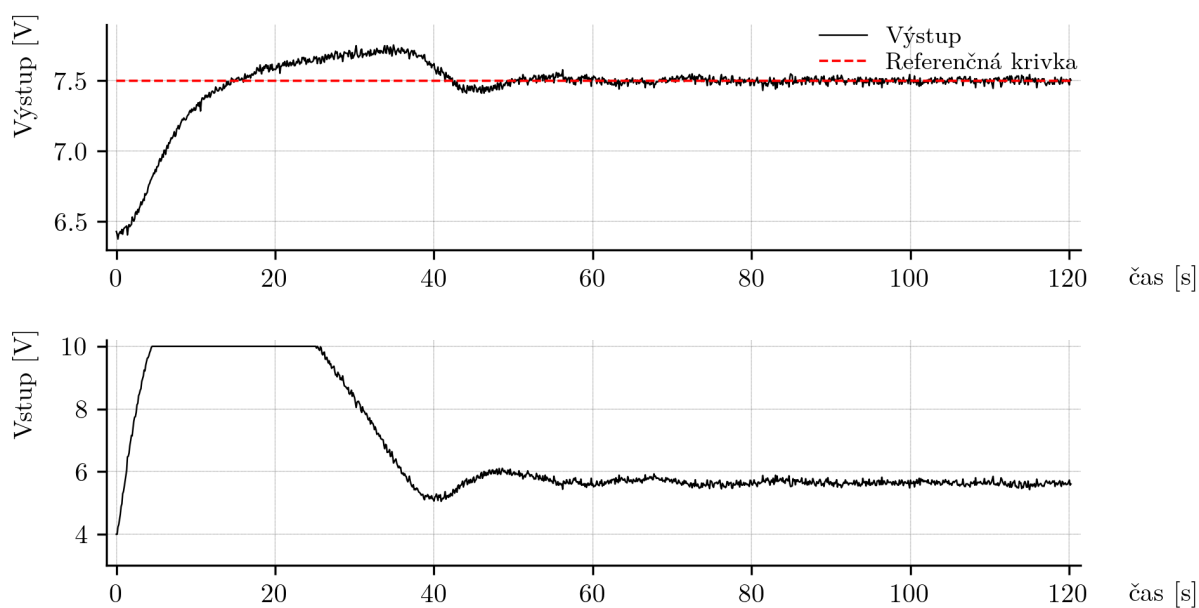
Kvalita regulácie bola hodnotená podľa troch kritérií:

- prekmit: žiadaná hodnota bola 0.35 V, maximálna hodnota výstupu dosiahla približne 0.52 V, čo zodpovedá približne 49.0 percentnému prekmitu,
- čas ustálenia: výstup sa ustálil približne po 28 sekundách,
- trvalá regulačná odchýlka: prakticky nulová.

Pri porovnaní so simuláciou uvedenou v predchádzajúcich kapitolách možno vidieť, že prekmit je výrazne vyšší. Dôvodom je skutočnosť, že identifikované modely nedokázali zachytiť celú dynamiku systému TS05 a viacero pomalých javov, ktoré sa prejavujú až pri reálnej prevádzke. Napriek tomu je regulačný čas veľmi priaznivý vzhľadom na to, že ide o pomalý tepelný systém. Rýchla regulácia bola prioritou, preto možno výsledok považovať za uspokojivý pre riadenie okolo jedného pracovného bodu.

Experiment 2: Reakcia na väčší skok

Nasledoval druhý experiment, ktorého cieľom bolo overiť správanie regulátora pri väčšej skokovej zmene žiadanej hodnoty, ktorá má simulovať prechod medzi rôznymi pracovnými bodmi systému. Systém bol najprv ustálený v pracovnom bode s výstupnou hodnotou približne 6.4 V a následne bola žiadaná hodnota skokovo zmenená na 7.5 V. Výsledok je zobrazený na obrázku 20.



Obr. 20: Experiment PI regulátora pri skoku z 6.4 V na 7.5 V

Z grafu je zrejmé, že počas regulácie došlo k výraznému presýteniu akčného zásahu. Ovládací signál dosiahol svoj limit a počas určitého času na ňom zotrval. Aj keď bola požadovaná hodnota dosiahnutá, takéto správanie je z pohľadu bezpečnosti a kvality regulácie nevyhovujúce.

Z tohto dôvodu možno konštatovať, že navrhnutý PI regulátor je vhodný na riadenie v blízkosti jedného pracovného bodu, avšak nie je vhodný na prepínanie medzi vzdialenejšími pracovnými bodmi. V ďalšej časti práce bude preto implementovaný postup umožňujúci bezpečné a stabilné prepínanie medzi pracovnými bodmi a následne budú jednotlivé regulátory skombinované tak, aby systém disponoval širším využiteľným rozsahom riadenia.

4 PI regulácia s prepínaním pracovných bodov

4.1 Prepínač pracovných bodov

V tejto časti sa budeme zaoberať návrhom a testovaním prepínača pracovných bodov. Prepínač bude predstavovať pomalý regulátor, ktorého úlohou bude zabezpečiť robustné prechody medzi rôznymi pracovnými bodmi systému TS05. Cieľom je vytvoriť taký spôsob riadenia, ktorý umožní spoľahlivý presun z ľubovoľného pracovného bodu do ľubovoľného iného pracovného bodu bez neželaných dynamických javov, ako je presýtenie akčného zásahu alebo príliš veľké prekmitové správanie.

Okrem samotného prepínača bola v tejto časti rozšírená aj pracovná oblasť regulácie o dva ďalšie pracovné body systému – pre napätia špirály 2 V a 6 V. Pre každý z týchto pracovných bodov bol postup opakovaný presne podľa postupu uvedeného v sekcii 2.2, teda boli:

- vykonané merania reakcií systému na jednotkové skoky ± 0.5 V okolo pracovného bodu,
- z nameraných údajov bola identifikovaná prenosová funkcia v danom pracovnom bode,
- pre každý pracovný bod bol navrhnutý PI regulátor s vlastnými parametrami,
- boli určené hodnoty y_{pb} a rozsah výstupu pre ± 0.5 V zmenu vstupu.

Prehľad všetkých troch pracovných bodov je uvedený v tabuľke 2, kde sú uvedené identifikované prenosové funkcie, hodnoty pracovného bodu y_{pb} a parametre jednotlivých PI regulátorov.

Tabuľka 2: Parametre pracovných bodov, identifikované prenosové funkcie a PI regulátory

u_{pb}	Prenosová funkcia	y_{pb}	Δy_{pb} pri $u_{pb} \pm 0.5$ V	PI parametre
2 V	$\frac{5.79}{s^2 + 44.21s + 7.276}$	4.4884	± 0.41	$K_p = 2.77, K_i = 1.48$
4 V	$\frac{6.074}{s^2 + 59.01s + 8.95}$	6.3412	± 0.35	$K_p = 3.58, K_i = 1.84$
6 V	$\frac{7.161}{s^2 + 52.87s + 21.37}$	7.7441	± 0.18	$K_p = 1.83, K_i = 2.20$

Keďže pôvodné statické charakteristiky systému boli merané ešte počas letného obdobia, bolo rozhodnuté hodnoty y_{pb} pre tieto experimenty zmerať znova, rovnakým spôsobom ako v sekcii 2.1.2. V tabuľke sú preto uvedené aktuálne hodnoty pracovných bodov, ktoré vernejšie reprezentujú stav systému počas meraní v zimnom období.

4.1.1 Opis a schéma riadenia z použitím prepínača

V tejto časti sa prezentuje návrh riadenia systému TS05 s využitím prepínača pracovných bodov. Zámerom riadenia je pokryť dve odlišné situácie:

- reguláciu v okolí zvoleného pracovného bodu.
- bezpečný prechod medzi pracovnými bodmi bez neželaných javov, ako je dlhodobé presýtenie akčného zásahu alebo príliš agresívna dynamika.

Navrhnutá štruktúra je založená na tom, že pri zmene globálnej žiadanej hodnoty má prioritu prechodový režim (prepínač pracovných bodov), a až po jeho ukončení sa aktivuje lokálna PI regulácia v okolí aktuálneho pracovného bodu.

Bloková schéma logickej postupnosti operácií navrhnutého algoritmu riadenia je znázornená na obr. 21.

Ako už bolo uvedené, pokiaľ nedôjde ku skokovej zmene globálnej žiadanej hodnoty, systém pracuje v režime lokálnej regulácie v okolí aktuálneho pracovného bodu. V tomto

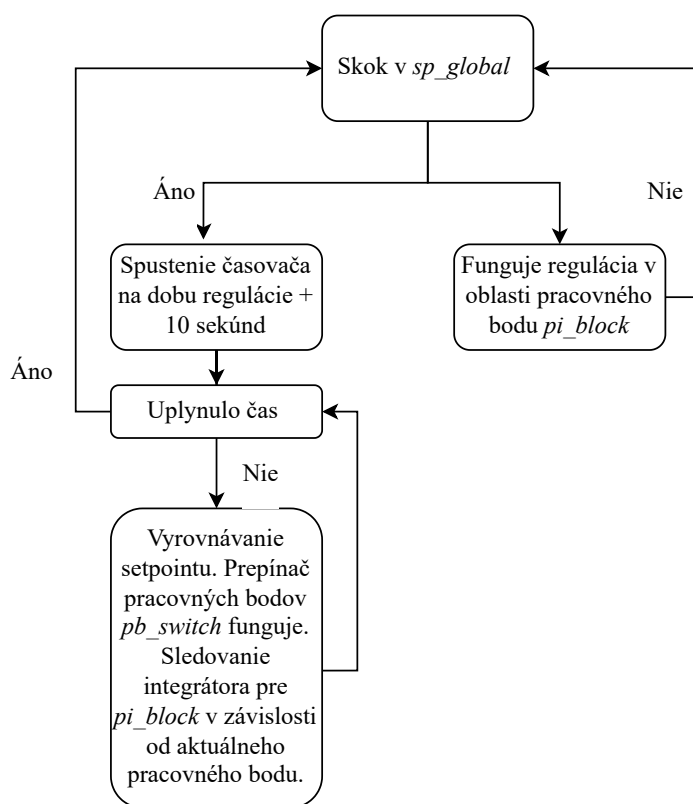
prípade je aktívny PI regulátor, ktorý zabezpečuje reguláciu nad normalizovanými veličinami

$$y_N = y - y_{pb}, \quad u_N = u - u_{pb},$$

kde y_{pb} a u_{pb} predstavujú posuny výstupnej a vstupnej veličiny príslušného pracovného bodu.

Informáciu o tom, v ktorom pracovnom bode sa systém nachádza, nepretržite poskytuje *solver*. Táto informácia slúži lokálnemu regulátoru na výber zodpovedajúcich posunov y_{pb} , u_{pb} a zároveň na nastavenie parametrov PI regulátora K_p a K_i .

V prípade, že dôjde ku skokovej zmene globálnej žiadanej hodnoty, lokálna PI regulácia je dočasne deaktivovaná a riadenie preberá prepínač pracovných bodov. Ten zabezpečuje plynulý prechod medzi pracovnými bodmi a po jeho ukončení sa riadenie automaticky prepne späť do režimu lokálnej regulácie v okolí nového pracovného bodu.

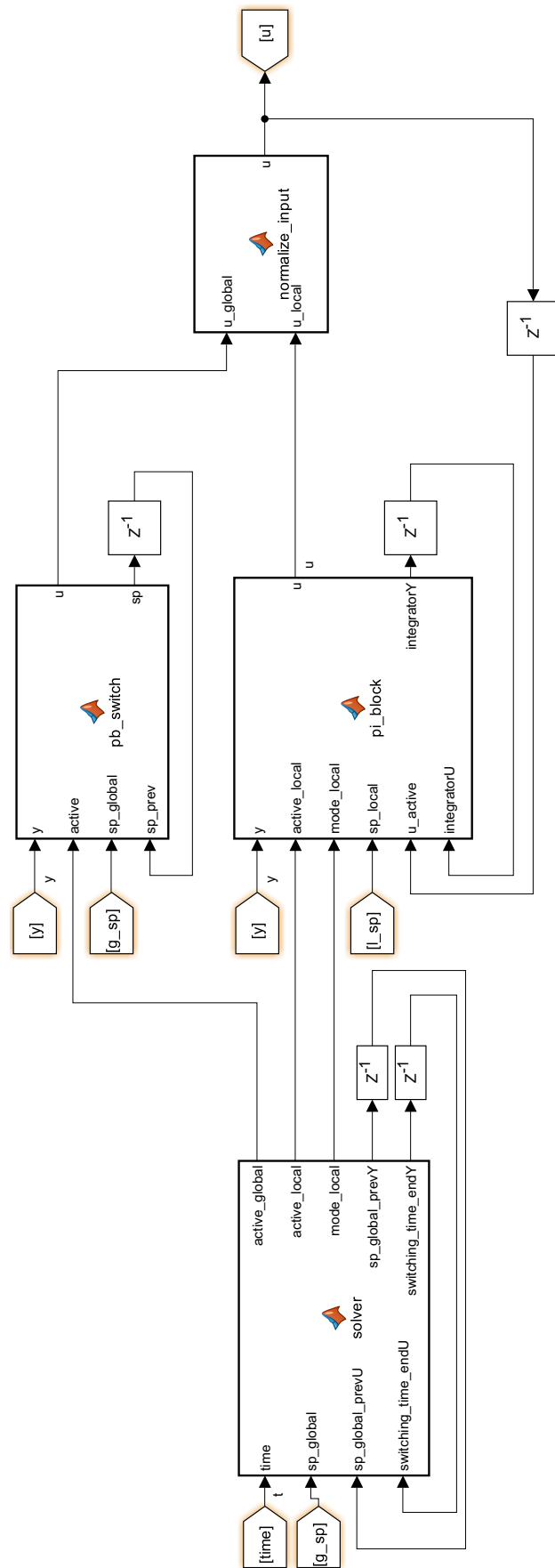


Obr. 21: Bloková schéma logickej postupnosti operácií riadenia systému TS05

Schéma celého riadenia je uvedená na obr. 22.

Ako je možné vidieť na schéme, rozhodovacia logika je sústredená v bloku *solver*, ktorý na základe globálnej žiadanej hodnoty určuje, či sa systém nachádza v režime prechodu medzi pracovnými bodmi, alebo už môže prebiehať lokálna regulácia okolo pracovného bodu. Okrem toho blok *solver* nepretržite určuje aj aktuálny režim *mode_local*, t. j. informáciu o tom, ku ktorému z pracovných bodov (2 V, 4 V alebo 6 V na špirále) má byť priradená lokálna regulácia.

Implementácia bloku *solver* je uvedená v kóde 5. Základom je výpočet času, počas ktorého má byť aktívny prechodový režim. Tento čas sa odvodzuje z veľkosti zmeny globálnej žiadanej hodnoty a z kroku zjemnenia *step_to_soft*, pričom sa uvažuje aj dodatočná časová rezerva *extra_time*.



Obr. 22: Schéma riadenia systému TS05 s prepínačom pracovných bodov

```

1 function [active_global, active_local, mode_local, sp_global_prevY,
   switching_time_endY] = solver(time, sp_global, sp_global_prevU,
   switching_time_endU)
2
3 %Constants
4 step_to_soft = 0.005;
5 Ts = 0.1;
6 extra_time = 10;
7 eps = 1e-5;
8
9 active_global = 0;
10 active_local = 0;
11
12 %Time of switch working
13 if abs(sp_global - sp_global_prevU) > eps
14     reg_time = (abs(sp_global - sp_global_prevU) / step_to_soft) * Ts + extra_time;
15     reg_time = max(reg_time, extra_time);
16     switching_time_endY = time + reg_time;
17 else
18     switching_time_endY = switching_time_endU;
19 end
20
21 %Activators of PI and switch
22 if time < switching_time_endY
23     active_global = 1;
24     active_local = 0;
25 else
26     active_global = 0;
27     active_local = 1;
28 end
29
30 if sp_global > 7.6
31     mode_local = 6;
32 elseif sp_global > 6.2
33     mode_local = 4;
34 else
35     mode_local = 2;
36 end
37
38 %memory
39 sp_global_prevY = sp_global;
40
41 end

```

Výpis kódu 5: MATLAB implementácia rozhodovacej logiky prepínača pracovných bodov (*solver*)

Logika kódu 5 vyplýva z výrazu

$$t_{\text{reg}} = \left(\frac{|sp_{\text{global}} - sp_{\text{global,prev}}|}{\Delta sp} \right) T_s + t_{\text{extra}}, \quad t_{\text{end}} = t + t_{\text{reg}}, \quad (54)$$

kde Δsp predstavuje krok zjemnenia (*step_to_soft*), T_s je vzorkovací čas a t_{extra} je časová rezerva. Prechodový režim má v navrhutej štruktúre prioritu, t.j. počas jeho trvania je lokálna PI regulácia zablokovaná.

Po rozhodnutí o režime nasledujú samotné regulátory. V ľubovoľnom časovom okamihu je aktívny vždy iba jeden z nich: buď prechodový regulátor (prepínač pracovných bodov), alebo lokálny PI regulátor. Prepínač pracovných bodov *pb_switch* realizuje zjemnenie žiadanej hodnoty tak, aby namiesto skoku vznikla rampa, čo výrazne znižuje riziko presýtenia akčného zásahu pri veľkých zmenách referencie. Zjemnenie prebieha s krokom $\Delta sp = 0.005$ na jednu iteráciu, t.j. požadovaná hodnota sa mení postupne, kým nedosiahne cieľovú globálnu hodnotu. Zjemnený setpoint je následne použitý v jednoduchom P regulátore.

Kód bloku *pb_switch* je uvedený v kóde 6.

```

1 function [u, sp] = pb_switch(y, active, sp_global, sp_prev)
2
3 %Constants
4 Kp = 4;

```



```

5 Ts = 0.1;
6 step_to_soft = 0.005;
7
8 %Logic
9 if active
10
11     if abs(sp_global - sp_prev) > step_to_soft
12         delta = sign(sp_global - sp_prev) * step_to_soft;
13     else
14         delta = sp_global - sp_prev;
15     end
16
17     sp = sp_prev + delta;
18     u = Kp * (sp - y);
19
20 else
21
22     sp = sp_global;
23     u = 0;
24
25 end
26
27 u = min(max(u,0), 10);
28
29 end

```

Výpis kódu 6: MATLAB implementácia prepínača pracovných bodov (*pb_switch*)

Čas, počas ktorého je *pb_switch* aktívny, je priamo viazaný na veľkosť zmeny globálneho setpointu, čo bolo definované v bloku *solver* vzťahom (54). Po uplynutí tejto doby sa aktivuje lokálny PI regulátor, ktorý zabezpečuje presné sledovanie žiadanej hodnoty v okolí pracovného bodu.

Lokálna PI regulácia je realizovaná blokom *pi_block*, ktorý obsahuje dve dôležité úpravy:

- ziskové plánovanie (gain scheduling) podľa aktuálne zvoleného pracovného bodu,
- sledovanie integrátora aj v neaktívnom stave (tzv. bumpless transfer), aby po opätovnej aktivácii nevznikol neželaný skok akčného zásahu.

Ziskové plánovanie spočíva v tom, že podľa signálu *mode_local* (určeného v *solver*) sa zvolia príslušné parametre K_p , K_i a hodnoty pracovného bodu (u_{pb} , y_{pb}). Normalizácia prebieha odčítaním y_{pb} od meraného výstupu a akčný zásah je limitovaný v rozsahu okolo u_{pb} .

Implementácia *pi_block* je uvedená v kóde 7.

```

1 function [u, integratorY] = pi_block(y, active_local, mode_local, sp_local, u_active,
2     integratorU)
3
4 Ts = 0.1;
5
6 %Gain scheduling
7 if mode_local == 2
8     pb_u = 2; pb_y = 4.4884;
9     Kp = 2.77; Ki = 1.48;
10 elseif mode_local == 4
11     pb_u = 4; pb_y = 6.3412;
12     Kp = 3.58; Ki = 1.84;
13 else
14     pb_u = 6; pb_y = 7.7441;
15     Kp = 1.83; Ki = 2.20;
16 end
17
18 u_min = 0 - pb_u;
19 u_max = 10 - pb_u;
20
21 yN = y - pb_y;
22 e = sp_local - yN;
23
24 if active_local == 0

```

```

24     uN = u_active - pb_u;
25     uN = min(max(uN, u_min), u_max);
26     integratorY = (uN - Kp*e) / Ki;
27     u = 0;
28 else
29     integratorY = integratorU + e*Ts;
30     uN = Kp*e + Ki*integratorY;
31     uN = min(max(uN, u_min), u_max);
32     u = uN + pb_u;
33 end
34
35 end

```

Výpis kódu 7: MATLAB implementácia lokálnej PI regulácie so ziskovým plánovaním a sledovaním integrátora (*pi_block*)

Sledovanie integrátora v neaktívnom stave je realizované tak, že sa z rovnice PI regulátora v normalizovanom tvare

$$u_N = K_p e + K_i I \quad (55)$$

vyjadrí stav integrátora

$$I = \frac{u_N - K_p e}{K_i}. \quad (56)$$

Týmto spôsobom je zabezpečené, že pri prepnutí režimu nedôjde k náhlej zmene vnútornej integrujúcej zložky, a teda ani k nežiadanejmu skoku na výstupe regulátora.

Výstupy prechodového regulátora a lokálneho PI regulátora sa následne kombinujú v bloku *normalize_input*, kde sa vykoná súčet a finálne obmedzenie signálu do rozsahu 0–10 V. Keďže v každom okamihu je aktívny iba jeden regulátor (druhý dáva nulový výstup), je táto forma kombinácie v danej štruktúre prípustná a nevedie k nejednoznačnosti riadenia.

Kód bloku *normalize_input* je uvedený v kóde 8.

```

1 function u = normalize_input(u_global, u_local)
2
3     u_sum = u_global + u_local;
4     u = min(max(u_sum, 0), 10);
5
6 end

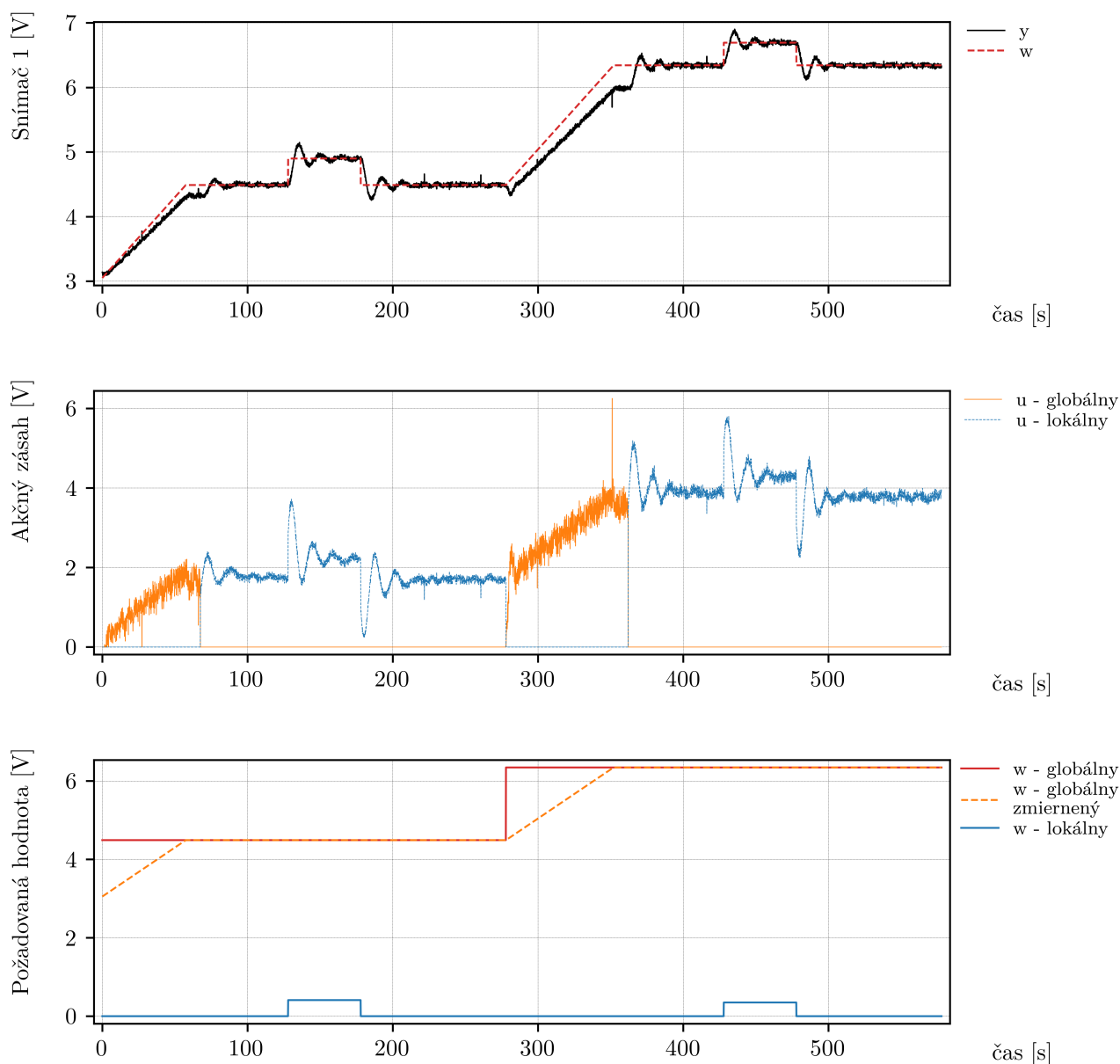
```

Výpis kódu 8: MATLAB implementácia finálnej normalizácie a obmedzenia akčného zásahu (*normalize_input*)

4.1.2 Experiment s prepínaním pracovných bodov

V tejto podsekcii bol uskutočnený experiment na reálnom systéme TS05, ktorého cieľom bolo overiť správnu funkciu navrhnutého algoritmu riadenia pri prepínaní medzi pracovnými bodmi. Experiment pozostával z dvoch hlavných fáz. Najskôr bol systém regulovaný v okolí pracovného bodu pri napätí špirály 4 V, kde bola realizovaná lokálna skoková zmena žiadanej hodnoty. Následne došlo k prechodu na pracovný bod 6 V, po ktorom bola opäť aplikovaná lokálna skoková zmena referencie.

Priebeh experimentu je znázornený na obr. 23, kde sú zobrazené tri podgrafy. Prvý podgraf zobrazuje výstupnú veličinu systému spolu so žiadanou hodnotou, pričom žiadaná hodnota je v tomto prípade normalizovaná za účelom prehľadnejšej vizualizácie priebehu regulácie. Skutočné (nenormalizované) hodnoty globálneho a lokálneho setpointu sú uvedené v treťom podgrafe.



Obr. 23: Experiment s prepínaním pracovných bodov: výstup systému, akčné zásahy a žiadané hodnoty

Z priebehu akčného zásahu na druhom podgrafe je zrejmé, že riadiaci signál počas celého experimentu nenaráža na svoje maximálne obmedzenie, čo svedčí o vhodne zvolenej stratégii regulácie. Zároveň je možné pozorovať, že v každom okamihu je aktívny iba jeden akčný zásah: ak pracuje lokálny PI regulátor, výstup prepínača pracovných bodov je nulový a naopak. Týmto sa potvrdzuje správna sekvenčná činnosť jednotlivých blokov riadenia, presne v súlade s návrhom algoritmu.

Dôležitým pozorovaním je aj správanie systému pri prepnutí z prechodového režimu

na lokálnu PI reguláciu. Po aktivácii lokálneho regulátora nedochádza k náhlemu „výstrelu“ akčného zásahu, čo indikuje správnu funkciu sledovania stavu integrátora. Integrujúca zložka regulátora je teda pri prepínaní režimov v konzistentnom stave, čo výrazne zvyšuje kvalitu prechodu medzi režimami riadenia.

V regulácii sa súčasne využíva lokálny setpoint a zjemnený globálny setpoint generovaný prepínačom pracovných bodov.

Napriek celkovo uspokojivému správaniu systému sa počas prepínania pracovných bodov objavuje aj jeden nežiaduci jav. Keďže pred samotným prepnutím pracuje lokálny PI regulátor, po aktivácii prepínača pracovných bodov je akčný zásah prepínača v počiatočnom okamihu nulový a následne pomerne rýchlo narastá. Tento prechodný jav spôsobí krátkodobý pokles výstupnej veličiny systému, ktorý je síce časovo obmedzený, avšak pozorovateľný v nameraných dátach. Zároveň je možné pozorovať, že keďže prepínač pracovných bodov je realizovaný vo forme P regulátora, v jeho činnosti sa vyskytuje malá trvalá regulačná odchýlka. Tento nedostatok by bolo možné v budúcnosti odstrániť rozšírením regulátora o integračnú zložku.

V ďalšej fáze práce budú tieto problémy riešené a navrhnutý algoritmus riadenia bude implementovaný na ďalších platformách.

5 MRAC regulácia – gradientná metóda

V tejto kapitole je riešená adaptívna regulácia typu MRAC (Model Reference Adaptive Control), ktorá umožňuje automatické prispôsobovanie parametrov regulátora pri nepresne známej dynamike riadeného systému. Na rozdiel od klasických regulátorov s pevnými parametrami MRAC využíva referenčný model, ktorý definuje požadované správanie uzavretého regulačného obvodu, a adaptačný mechanizmus, ktorý zabezpečuje sledovanie tohto správania v čase.

V ďalšej časti je popísaný samotný algoritmus gradientnej MRAC regulácie, vrátane voľby referenčného modelu, štruktúry zákona riadenia a adaptačného zákona založeného na gradientnej metóde.

5.1 Popis algoritmu

Gradientná MRAC regulácia je založená na myšlienke, že dynamika riadeného systému má byť v čase prispôbovaná tak, aby výstup systému sledoval výstup zvoleného referenčného modelu. Návrh algoritmu pozostáva z troch základných častí: voľba referenčného modelu, návrh zákona riadenia a odvodenie adaptačného zákona.

Model riadeného systému

Uvažujme lineárny spojitý systém druhého rádu

$$G_s(s) = \frac{b_0}{s^2 + a_1s + a_0}, \quad (57)$$

kde a_0 , a_1 , b_0 sú neznáme konštanty. Predpokladá sa znalosť znamienka parametra b_0 .

Referenčný model a statické zosilnenie

Pri voľbe referenčného modelu je potrebné zabezpečiť, aby jeho statické zosilnenie bolo rovné jednej, t. j.

$$K = 1,$$

aby nedochádzalo k nechcenému zosilňovaniu referenčného signálu.

Pre systém druhého rádu zvolíme referenčný model

$$\frac{y_m(s)}{r(s)} = \frac{b_{0m}}{s^2 + a_{1m}s + a_{0m}}, \quad (58)$$

pričom sa volí

$$b_{0m} = a_{0m}, \quad (59)$$

čím sa zabezpečí jednotkové statické zosilnenie modelu.

Zákon riadenia

Všeobecný tvar zákona riadenia pri splnenej podmienke zhody je

$$u(t) = \Theta^T(t) \omega(t), \quad (60)$$

kde regresný vektor obsahuje merateľné signály.

Pre systém druhého rádu zvolíme

$$\omega(t) = \begin{bmatrix} y(t) \\ \dot{y}(t) \\ r(t) \end{bmatrix}. \quad (61)$$

Zákon riadenia potom nadobúda tvar

$$u(t) = \Theta_1 y(t) + \Theta_2 \dot{y}(t) + \Theta_3 r(t). \quad (62)$$

Podmienka zhody a ideálne parametre

Po dosadení (62) do systému (57) dostávame podmienku zhody

$$y(s) (s^2 + (a_1 - b_0\Theta_2)s + a_0 - b_0\Theta_1) = b_0\Theta_3 r(s). \quad (63)$$

Prenos uzavretého regulačného obvodu je

$$\frac{y(s)}{r(s)} = \frac{b_0\Theta_3}{s^2 + (a_1 - b_0\Theta_2)s + (a_0 - b_0\Theta_1)}. \quad (64)$$

Porovnaním s referenčným modelom získame ideálny vektor parametrov

$$\Theta_1^* = \frac{a_0 - a_{0m}}{b_0}, \quad (65)$$

$$\Theta_2^* = \frac{a_1 - a_{1m}}{b_0}, \quad (66)$$

$$\Theta_3^* = \frac{b_{0m}}{b_0}. \quad (67)$$

Adaptačná odchýlka

Adaptačná odchýlka je definovaná ako

$$e(t) = y(t) - y_m(t), \quad (68)$$

pričom cieľom regulácie je dosiahnuť

$$\lim_{t \rightarrow \infty} e(t) = 0. \quad (69)$$

Účelová funkcia a MIT pravidlo

Minimalizovaná je účelová funkcia

$$J(\Theta) = \frac{1}{2} e^2(t). \quad (70)$$

MIT pravidlo je formulované ako

$$\frac{d\Theta}{dt} = -\alpha \frac{\partial J}{\partial \Theta} = -\alpha e \frac{\partial e}{\partial \Theta}. \quad (71)$$

Keďže y_m nezávisí od Θ , platí

$$\frac{d\Theta}{dt} = -\alpha e \frac{\partial y}{\partial \Theta}. \quad (72)$$

Zákon adaptácie pre systém druhého rádu

Pre systém druhého rádu sú adaptačné zákony dané vzťahmi

$$\dot{\Theta}_1 = -\alpha_1 e \operatorname{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} y, \quad (73)$$

$$\dot{\Theta}_2 = -\alpha_2 e \operatorname{sign}(b_0) \frac{s}{s^2 + a_{1m}s + a_{0m}} y, \quad (74)$$

$$\dot{\Theta}_3 = -\alpha_3 e \operatorname{sign}(b_0) \frac{1}{s^2 + a_{1m}s + a_{0m}} r. \quad (75)$$

Člen s v adaptačnom zákone pre parameter Θ_2 zohľadňuje dynamický vplyv derivácie výstupu v zákone riadenia.

5.1.1 Prevod systému do stavového priestoru

Pre potreby diskkrétnej implementácie je vhodné preniesť opis riadeného systému z tvaru prenosovej funkcie do tvaru stavového modelu.

Uvažujme všeobecný lineárny systém druhého rádu

$$G_s(s) = \frac{Y(s)}{U(s)} = \frac{b_1 s + b_0}{s^2 + a_1 s + a_0}. \quad (76)$$

Tento prenos je možné interpretovať ako kaskádové spojenie dvoch čiastkových prenosov zavedením pomocnej premennej $Z(s)$

$$\frac{Z(s)}{U(s)} = \frac{1}{s^2 + a_1 s + a_0}, \quad (77)$$

$$\frac{Y(s)}{Z(s)} = b_1 s + b_0. \quad (78)$$

Platí teda

$$\frac{Y(s)}{U(s)} = \frac{Y(s)}{Z(s)} \frac{Z(s)}{U(s)}. \quad (79)$$

Prvý z uvedených prenosov (77) možno zapísať v časovej oblasti ako diferenciálnu rovnicu

$$\ddot{z}(t) + a_1 \dot{z}(t) + a_0 z(t) = u(t). \quad (80)$$

Táto rovnica je druhého rádu a je možné ju previesť na sústavu diferenciálnych rovníc prvého rádu vhodnou voľbou stavových veličín.

Zavedme prvú stavovú veličinu

$$x_1(t) = z(t), \quad (81)$$

z čoho vyplýva

$$\dot{x}_1(t) = \dot{z}(t). \quad (82)$$

Ako druhú stavovú veličinu zvolíme

$$x_2(t) = \dot{z}(t), \quad (83)$$

a teda

$$\dot{x}_2(t) = \ddot{z}(t). \quad (84)$$

Z rovnice (80) je možné vyjadriť druhú deriváciu

$$\ddot{z}(t) = -a_1 \dot{z}(t) - a_0 z(t) + u(t), \quad (85)$$

čo po dosadení za stavové premenné vedie k

$$\dot{x}_2(t) = -a_1 x_2(t) - a_0 x_1(t) + u(t). \quad (86)$$

Sústava diferenciálnych rovníc prvého rádu má teda tvar

$$\dot{x}_1(t) = x_2(t), \quad (87)$$

$$\dot{x}_2(t) = -a_1 x_2(t) - a_0 x_1(t) + u(t). \quad (88)$$

V maticovom zápise možno systém vyjadriť ako

$$\dot{x}(t) = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix} x(t) + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u(t), \quad (89)$$

kde stavový vektor je definovaný ako

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}. \quad (90)$$

Výstupná rovnica vyplýva z prenosu (78). V časovej oblasti platí

$$y(t) = b_1 \dot{z}(t) + b_0 z(t). \quad (91)$$

Po vyjadrení pomocou stavových veličín dostávame

$$y(t) = b_1 x_2(t) + b_0 x_1(t), \quad (92)$$

resp. v maticovom tvare

$$y(t) = [b_0 \quad b_1] x(t). \quad (93)$$

Celý systém je teda v štandardnom stavovom tvare

$$\dot{x}(t) = Ax(t) + bu(t), \quad (94)$$

$$y(t) = c^T x(t), \quad (95)$$

Celý systém je teda v štandardnom stavovom tvare

$$\dot{x}(t) = Ax(t) + bu(t), \quad (96)$$

$$y(t) = c^T x(t), \quad (97)$$

kde jednotlivé matice majú tvar

$$A = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}. \quad (98)$$

Zavedená pomocná premenná $z(t)$ predstavuje interný stav systému, ktorý bol použitý na rozklad prenosovej funkcie do kaskádového tvaru. Pri zvolenej realizácii však výstup systému vzniká priamo ako lineárna kombinácia premenných $z(t)$ a $\dot{z}(t)$.

Z tohto dôvodu možno bez straty všeobecnosti interpretovať stavový vektor ako

$$x(t) = \begin{bmatrix} y(t) \\ \dot{y}(t) \end{bmatrix}, \quad (99)$$

keďže prvá stavová veličina reprezentuje samotný výstup systému a druhá jeho časovú deriváciu. Táto voľba je prirodzená pre systém druhého rádu a priamo nadväzuje na štruktúru diferenciálnej rovnice.

Stavový opis referenčného modelu

Referenčný model definovaný prenosom (58) je potrebné previesť do stavového tvaru.

Zvolíme stavový vektor

$$x_{ym}(t) = \begin{bmatrix} y_m(t) \\ \dot{y}_m(t) \end{bmatrix}. \quad (100)$$

Na základe všeobecného tvaru odvodeného v predchádzajúcej časti má referenčný model podobu

$$\dot{x}_{ym}(t) = A_{ym} x_{ym}(t) + b_{ym} r(t), \quad (101)$$

$$y_m(t) = c_{ym}^T x_{ym}(t), \quad (102)$$

kde

$$A_{ym} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{ym} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{ym} = \begin{bmatrix} b_{0m} \\ 0 \end{bmatrix}. \quad (103)$$

Stavový opis filtra výstupu

Filtračný člen použitý v adaptačnom zákone pre parameter Θ_1 (73) je daný prenosom

$$\frac{y_f(s)}{y(s)} = \frac{1}{s^2 + a_{1m}s + a_{0m}}. \quad (104)$$

Zvolíme stavový vektor

$$x_{yf}(t) = \begin{bmatrix} y_f(t) \\ \dot{y}_f(t) \end{bmatrix}. \quad (105)$$

Stavový opis má tvar

$$\dot{x}_{yf}(t) = A_{yf}x_{yf}(t) + b_{yf}y(t), \quad (106)$$

$$y_f(t) = c_{yf}^T x_{yf}(t), \quad (107)$$

kde

$$A_{yf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{yf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{yf} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (108)$$

Stavový opis filtra derivácie výstupu

V adaptačnom zákone pre parameter Θ_2 (74) sa nachádza člen

$$\frac{\dot{y}_f(s)}{y(s)} = \frac{s}{s^2 + a_{1m}s + a_{0m}}, \quad (109)$$

ktorý zodpovedá derivácii výstupu predchádzajúceho filtra.

Použijeme rovnaký stavový vektor $x_{yf}(t)$, pričom výstupná rovnica reprezentuje druhú stavovú veličinu

$$\dot{x}_{yf}(t) = A_{ydf}x_{yf}(t) + b_{ydf}y(t), \quad (110)$$

$$\dot{y}_f(t) = c_{ydf}^T x_{yf}(t), \quad (111)$$

kde

$$A_{ydf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{ydf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{ydf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (112)$$

Stavový opis filtra referencie

Filtračný člen použitý v adaptačnom zákone pre parameter Θ_3 (75) je daný prenosom

$$\frac{r_f(s)}{r(s)} = \frac{1}{s^2 + a_{1m}s + a_{0m}}. \quad (113)$$

Zvolíme stavový vektor

$$x_{rf}(t) = \begin{bmatrix} r_f(t) \\ \dot{r}_f(t) \end{bmatrix}. \quad (114)$$

Stavový opis má tvar

$$\dot{x}_{rf}(t) = A_{rf}x_{rf}(t) + b_{rf}r(t), \quad (115)$$

$$r_f(t) = c_{rf}^T x_{rf}(t), \quad (116)$$

kde

$$A_{rf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad b_{rf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad c_{rf} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}. \quad (117)$$

Zhrnutie štruktúry dynamických blokov

Zo získaných stavových opisov vyplýva, že všetky dynamické bloky využívajú rovnakú menovateľovú dynamiku, teda

$$A_{ym} = A_{yf} = A_{ydf} = A_{rf} = \begin{bmatrix} 0 & 1 \\ -a_{0m} & -a_{1m} \end{bmatrix}, \quad (118)$$

a rovnako

$$b_{ym} = b_{yf} = b_{ydf} = b_{rf} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (119)$$

Jednotlivé bloky sa odlišujú iba výstupnou maticou c .

Pre filter výstupu a jeho derivácie možno zapísať spoločnú výstupnú rovnicu v maticovom tvare

$$\begin{bmatrix} y_f(t) \\ \dot{y}_f(t) \end{bmatrix} = C_{yf} x_{yf}(t), \quad (120)$$

kde

$$C_{yf} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}. \quad (121)$$

Pre filter referencie platí

$$r_f(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} x_{rf}(t). \quad (122)$$

Referenčný model je realizovaný samostatne so stavovým vektorom $x_{ym}(t)$, pričom jeho výstup je

$$y_m(t) = \begin{bmatrix} b_{0m} & 0 \end{bmatrix} x_{ym}(t). \quad (123)$$

5.1.2 Pseudokód gradientnej MRAC regulácie (diskrétna realizácia)

Diskrétna realizácia gradientnej MRAC regulácie je vykonávaná v pevnom vzorkovacom intervale T_s . V každom vzorkovacom kroku sú k dispozícii aktuálne merané veličiny a stavové premenné uložené z predchádzajúceho kroku.

Zo systému je dostupný iba výstup $y(k)$. Keďže regulačný zákon využíva aj jeho časovú deriváciu, je potrebné túto veličinu numericky aproximovať. V diskretnom čase je derivácia výstupu určená pomocou spätnej diferencie

$$\dot{y}(k) = \frac{y(k) - y(k-1)}{T_s},$$

kde $y(k-1)$ predstavuje hodnotu výstupu z predchádzajúceho vzorkovacieho kroku.

Na základe regresného vektora $\omega(k) = [y(k), \dot{y}(k), r(k)]^T$ a aktuálneho vektora parametrov $\Theta(k)$ je vypočítaný regulačný zásah $u(k)$. Následne je určená chyba sledovania $e(k) = y(k) - y_m(k)$, kde $y_m(k)$ je výstup referenčného modelu.

Adaptačný zákon MIT pravidla je realizovaný v diskretnom tvare pomocou Eulerovej integrácie

$$\Theta(k+1) = \Theta(k) + T_s \dot{\Theta}(k),$$

pričom $\dot{\Theta}(k)$ vyplýva zo vzťahov (73) – (75) a obsahuje filtrované veličiny $y_f(k)$, $\dot{y}_f(k)$ a $r_f(k)$.

Stavové rovnice referenčného modelu a jednotlivých filtračných blokov sú integrované rovnakým spôsobom v tvare

$$x(k+1) = x(k) + T_s (Ax(k) + bv(k)),$$

kde vstup $v(k)$ predstavuje referenciu $r(k)$ alebo výstup systému $y(k)$ v závislosti od konkrétneho bloku.

Po vykonaní všetkých aktualizácií je hodnota $y(k)$ uložená pre použitie v nasledujúcom kroku pri výpočte numerickej derivácie.

Pseudokód diskretizovanej realizácie algoritmu je uvedený nižšie.

```

1
2 % Inputs:
3 % r(k) - reference signal
4 % y(k) - plant output
5 %
6 % States:
7 % x_ym(2x1) - [ym; ym_dot]
8 % x_yf(2x1) - [yf; ydot_f]
9 % x_rf - [rf; rf_dot]
10 % Theta - [Theta1; Theta2; Theta3]
11 % y_prev - y(k-1)
12 %
13 % Parameters:
14 % a0m, a1m, b0m
15 % alpha1, alpha2, alpha3
16 % sign_b0
17 % Ts
18
19 % Common dynamics
20 A = [ 0, 1;
21      -a0m, -a1m ];
22
23 b = [ 0;
24      1 ];
25
26 c_yf = [ 1, 0;
27         0, 1 ];
28
29 c_rf = [ 1, 0 ];
30
31 c_ym = [ b0m, 0 ];
32
33 % Differentiation
34 ydot = ( y - y_prev ) / Ts;
35
36 % Outputs from current states
37 ym = c_ym * x_ym; % ym(k)
38 yf_vec = c_yf * x_yf; % [yf(k); ydot_f(k)]
39 yf = yf_vec(1);
40 ydot_f = yf_vec(2);
41 rf = c_rf * x_rf; % rf(k)
42
43 % Tracking error
44 e = y - ym;
45
46 % Control law
47 omega = [ y; ydot; r ];
48 u = Theta.' * omega;
49
50 % MIT adaptation
51 dTheta = -sign_b0 * e * ...
52         [ alpha1 * yf;
53           alpha2 * ydot_f;
54           alpha3 * rf ];
55
56 Theta_next = Theta + Ts * dTheta;
57
58 % State updates
59 x_ym_next = x_ym + Ts * ( A * x_ym + [0; b0m] * r );
60 x_yf_next = x_yf + Ts * ( A * x_yf + b * y );
61 x_rf_next = x_rf + Ts * ( A * x_rf + b * r );
62
63 % Memory update
64 y_prev_next = y;

```

Výpis kódu 9: Pseudokód diskretnej implementácie gradientného MRAC regulátora

5.2 Implementácia MRAC v prostredí MATLAB

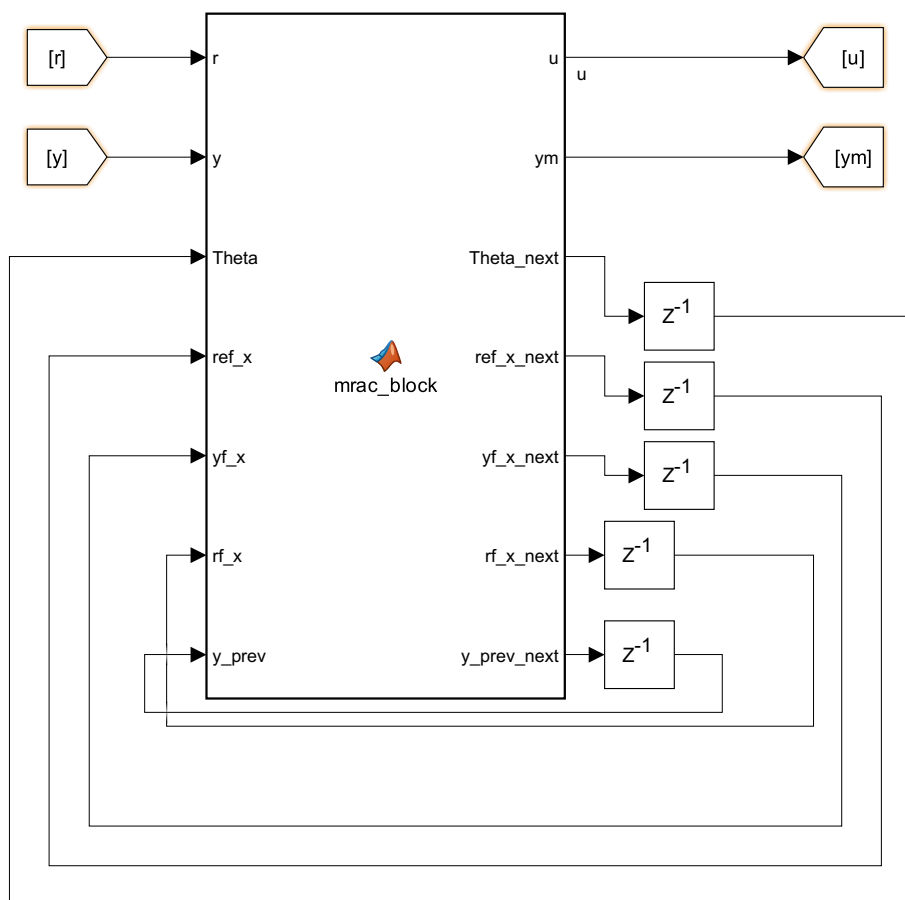
Cieľom tejto časti bolo implementovať navrhnutý gradientný MRAC algoritmus v prostredí MATLAB/Simulink bez využitia štandardných blokov riadenia, preddefinovaných adaptívnych štruktúr alebo iných pokročilých nástrojov Simulinku. Celý algoritmus je realizovaný výlučne v rámci jedného bloku MATLAB Function, pričom všetky výpočty – riadiaci zákon, referenčný model, filtračné členy aj adaptačný mechanizmus – sú explicitne implementované v zdrojovom kóde.

Použitý blok MATLAB Function nemá vnútornú pamäť medzi jednotlivými vzorkovacími krokmi. Z tohto dôvodu je potrebné zabezpečiť, aby všetky stavové premenné, ktoré musia byť prenesené do nasledujúceho diskretného kroku, boli vedené ako vstupno-výstupné premenné bloku. Konkrétne ide o:

- stav referenčného modelu x_m ,
- stav filtra výstupu x_{yf} ,
- stav filtra referencie x_{rf} ,
- adaptačný vektor parametrov Θ ,
- predchádzajúcu hodnotu výstupu systému y_{prev} .

Tieto veličiny sú na výstupe bloku aktualizované a následne spätne privedené na jeho vstup, čím sa zabezpečí diskretná dynamická realizácia celého algoritmu.

Schéma prepojenia blokov v prostredí Simulink je znázornená nižšie.



Obr. 24: Zapojenie MRAC regulátora v bloku MATLAB Function so spätnou väzbou stavov

Implementačný kód použitý v bloku MATLAB Function má nasledujúci tvar:

```
1
2 function [u, ym, Theta_next, ref_x_next, yf_x_next, rf_x_next, y_prev_next] ...
3     = mrac_block(r, y, Theta, ref_x, yf_x, rf_x, y_prev)
4
5 % constant parameters
6 a0m = 1;
7 a1m = 5;
8 b0m = 1;
9
10 alpha1 = 0.6;
11 alpha2 = 0.001;
12 alpha3 = 0.005;
13
14 sign_b0 = 1;
15 Ts = 0.1;
16
17 A = [0, 1;
18     -a0m, -a1m];
19
20 B_ref = [0;
21         b0m];
22
23 B_f = [0;
24       1];
25
26 % derivative approximation
27 ydot = (y - y_prev) / Ts;
28
29 % outputs from current states
30 ym = ref_x(1);
31 yf = yf_x(1);
32 yf_s = yf_x(2);
33 rf = rf_x(1);
34
35 % tracking error
36 e = y - ym;
37
38 % control law
39 omega = [y; ydot; r];
40 u = Theta.' * omega;
41
42 % MIT adaptation law
43 dTheta = -sign_b0 * e * ...
44     [alpha1 * yf;
45     alpha2 * yf_s;
46     alpha3 * rf];
47
48 Theta_next = Theta + Ts * dTheta;
49
50 % state updates (Euler method)
51 ref_x_next = ref_x + Ts * (A * ref_x + B_ref * r);
52 yf_x_next = yf_x + Ts * (A * yf_x + B_f * y);
53 rf_x_next = rf_x + Ts * (A * rf_x + B_f * r);
54
55 % memory update
56 y_prev_next = y;
57
58 end
```

Výpis kódu 10: Implementácia diskrétného MRAC regulátora v MATLAB Function bloku

Literatúra

- [1] M. Fikar, J. Mikleš, *Identifikácia systémov*. Slovenská technická univerzita v Bratislave, 1998. ISBN 80-227-1177-2.
- [2] M. Tárník, Učebný text KUT015: *Laboratórne zariadenie TS: orientačný prehľad*. Slovenská technická univerzita v Bratislave, 2025. Dostupné online: <https://github.com/OkoliePracovnehoBodu/KUT/>
- [3] D. C. Montgomery, E. A. Peck, G. G. Vining, *Introduction to Linear Regression Analysis, Fifth Edition*. John Wiley & Sons, Inc., Hoboken, New Jersey, 2012. ISBN 978-0-470-54281-1.